



Metaheuristic optimization

بهینه‌سازی فرا ابتکاری

Mohammad Khalooei

Spring 2026 — Amirkabir University of Technology





- **Mohammad Khalooei**

- PhD in Artificial Intelligence
- Lecturer at Sharif & Amirkabir university of technology



Khalooei@aut.ac.ir

mkhalooei90@gmail.com

@mkhalooei



هدف کلی

هدف این درس آشنایی و تولید الگوریتم‌های فراابتکاری برای حل مسایل مختلف بهینه‌سازی است. در این درس ابتدا الگوریتم‌های تکاملی معرفی می‌شوند و سپس تعدادی از الگوریتم‌های فراابتکاری شناخته شده معرفی می‌شوند و در نهایت ابزارهای لازم برای تولید انواع الگوریتم‌های فراابتکاری بررسی می‌شوند.

سرفصل‌ها

- تئوری ماکروسکوپی و میکروسکوپی تکامل
- الگوریتم‌های تکاملی: چارچوب کلی، عملگرهای تنوع، عملگرهای انتخاب
- الگوریتم‌های تکاملی: الگوریتم ژنتیکی، استراتژی تکاملی، برنامه‌نویسی ژنتیکی، الگوریتم ممتیک
- الگوریتم‌های جستجوی محلی: الگوریتم تپه‌نوردی، قدم زدن تصادفی، گرم و سرد کردن تدریجی
- الگوریتم‌های مبتنی بر جمعیت: گروه ذرات، کلونی مورچگان
- مدیریت محدودیت‌ها در بهینه‌سازی در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند قله‌ای در الگوریتم‌های فراابتکاری
- مسائل بهینه‌سازی چند هدفی در الگوریتم‌های فراابتکاری
- روش‌های بهینه‌سازی پارامترهای الگوریتم‌های فراابتکاری

روش ارزشیابی (پیشنهادی)

- تمرین‌ها و پروژه: ۲۰ درصد نمره
- آزمون‌های میان‌ترم و پایانی: ۶۰ درصد نمره
- پروژه پژوهشی: ۲۰ درصد نمره

منابع علمی پیشنهادی

- [1] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer, 2015.
- [2] K. A. D. Jong. *Evolutionary Computation: A Unified Approach*. MIT press, 2006.



عنوان درس به فارسی: بهینه‌سازی فرا ابتکاری		
عنوان درس به انگلیسی: Meta-Heuristic Optimization	نوع درس و واحد	
دروس پیش‌نیاز:	— پایه ☐ نظری ☒	
دروس هم‌نیاز:	— تخصصی الزامی ☒ عملی ☐	
تعداد واحد:	۳	تخصصی اختیاری ☐ نظری-عملی ☐
تعداد ساعت:	۴۸	مهارتی ☐

- **Lesson Objective:**

- This lesson aims to introduce various metaheuristic algorithms for solving different optimization problems. At the end of the lesson, students will become familiar with several well-known metaheuristic algorithms and will learn about the types of metaheuristics required for different optimization problems.

Main Topics:

- Introduction to Macro and Micro Evolutionary Computation
- Global, local, and population-based metaheuristics
- Single-objective and multi-objective metaheuristics
- Trajectory-based metaheuristics (such as Simulated Annealing, Tabu Search, etc.)
- Population-based metaheuristics (such as Genetic Algorithms, Particle Swarm Optimization, etc.)
- Constraint handling techniques in metaheuristic algorithms
- Several issues in improving the performance of metaheuristics
- Several target-oriented improvement issues in metaheuristics
- Methods for parameter tuning and adaptation in metaheuristic algorithms



Roadmap

Metaheuristic Optimization





- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?

- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search

- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search

- **Phase 3: Evolutionary and Population-Based Algorithms**

- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations



- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?



- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search

- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search

- **Phase 3: Evolutionary and Population-Based Algorithms**

- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations



- **Phase 0: Preliminaries of Optimizations**

- What is model?
- What are parameters of model?
- Parameter space vs Sample space
- What is loss function? What are they?
- What are Training of machine learning models?



- **Phase 1: Foundations of Optimization**

- Introduction to Optimization and Metaheuristics
- Modeling Optimization Problems
- Basic Concepts in Search

- **Phase 2: Single-Solution Metaheuristics**

- Random Search and Hill Climbing
- Simulated Annealing
- Tabu Search

- **Phase 3: Evolutionary and Population-Based Algorithms**

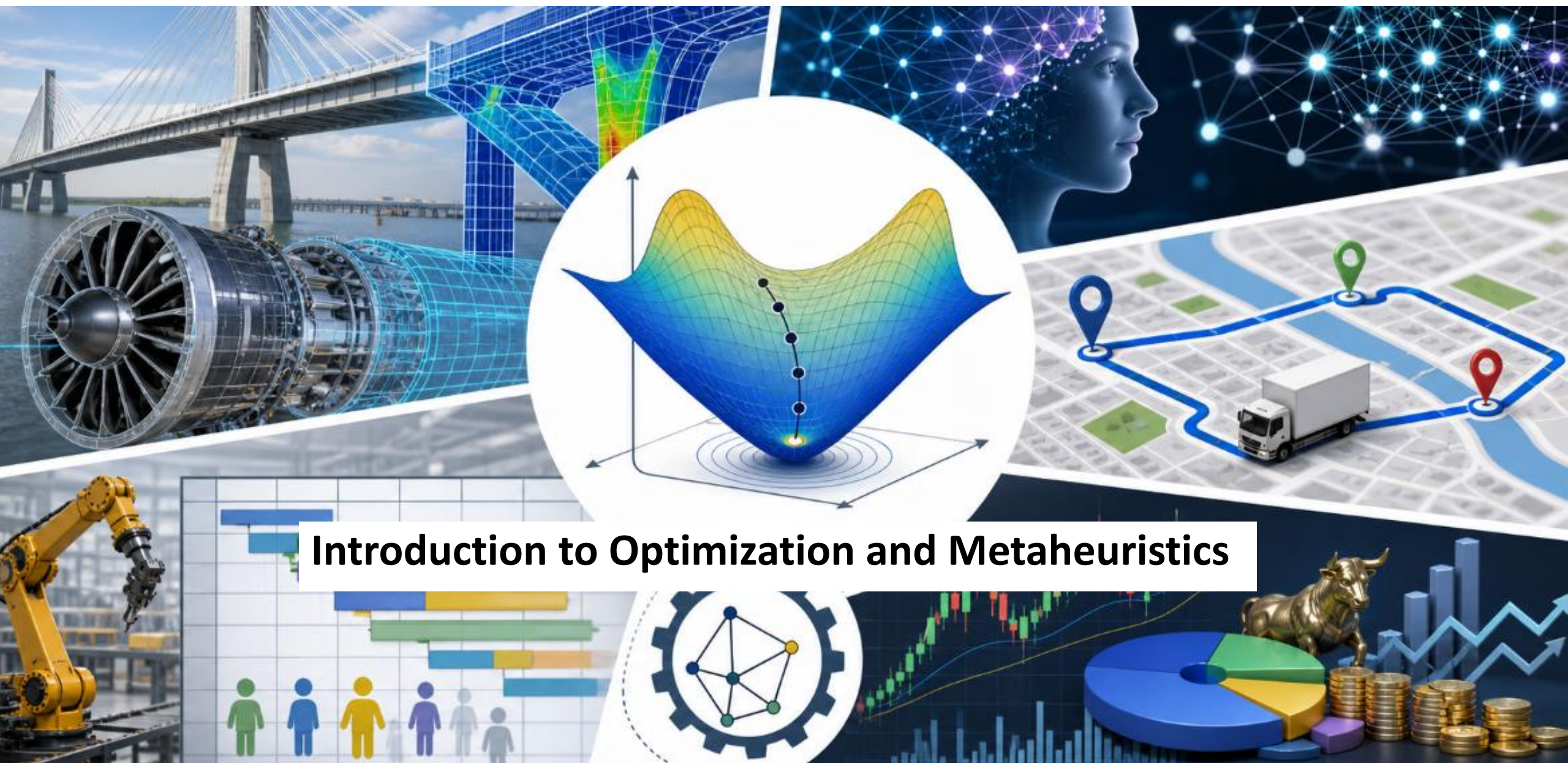
- Introduction to Evolutionary Computation
- Genetic Algorithm
- Evolution Strategies and Differential Evolution

- **Phase 4: Swarm Intelligence Algorithms**

- Particle Swarm Optimization
- Ant Colony Optimization
- Other Nature-Inspired Metaheuristics

- **Phase 5: Analysis, Comparison, and Advanced Topics**

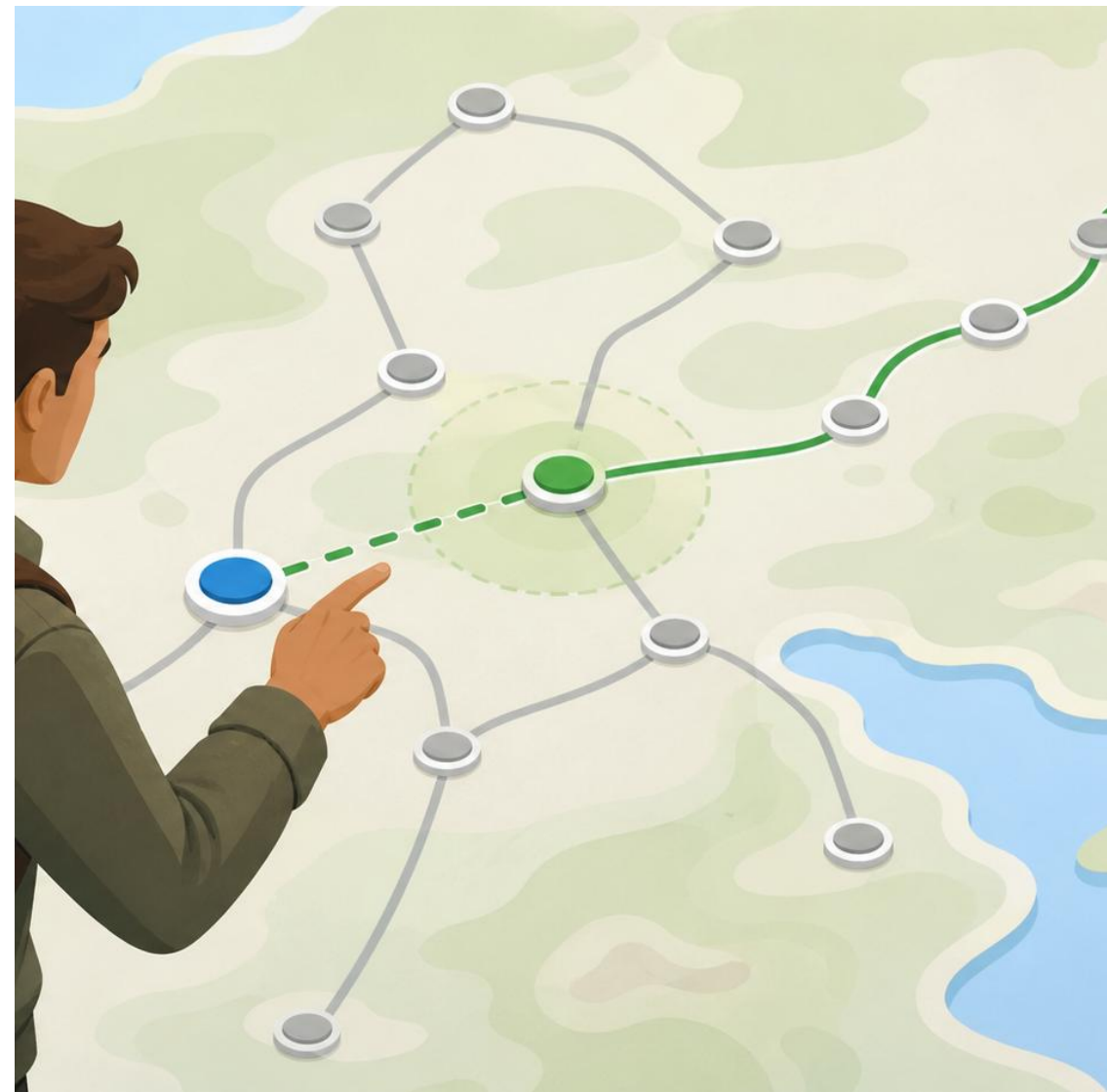
- Performance Evaluation of Metaheuristics
- Parameter Tuning and Experimental Design
- Hybrid Metaheuristics and Final Project Presentations



Introduction to Optimization and Metaheuristics



- Suppose you are planning a delivery route between 10 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?





- Suppose you are planning a delivery route between 10 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?

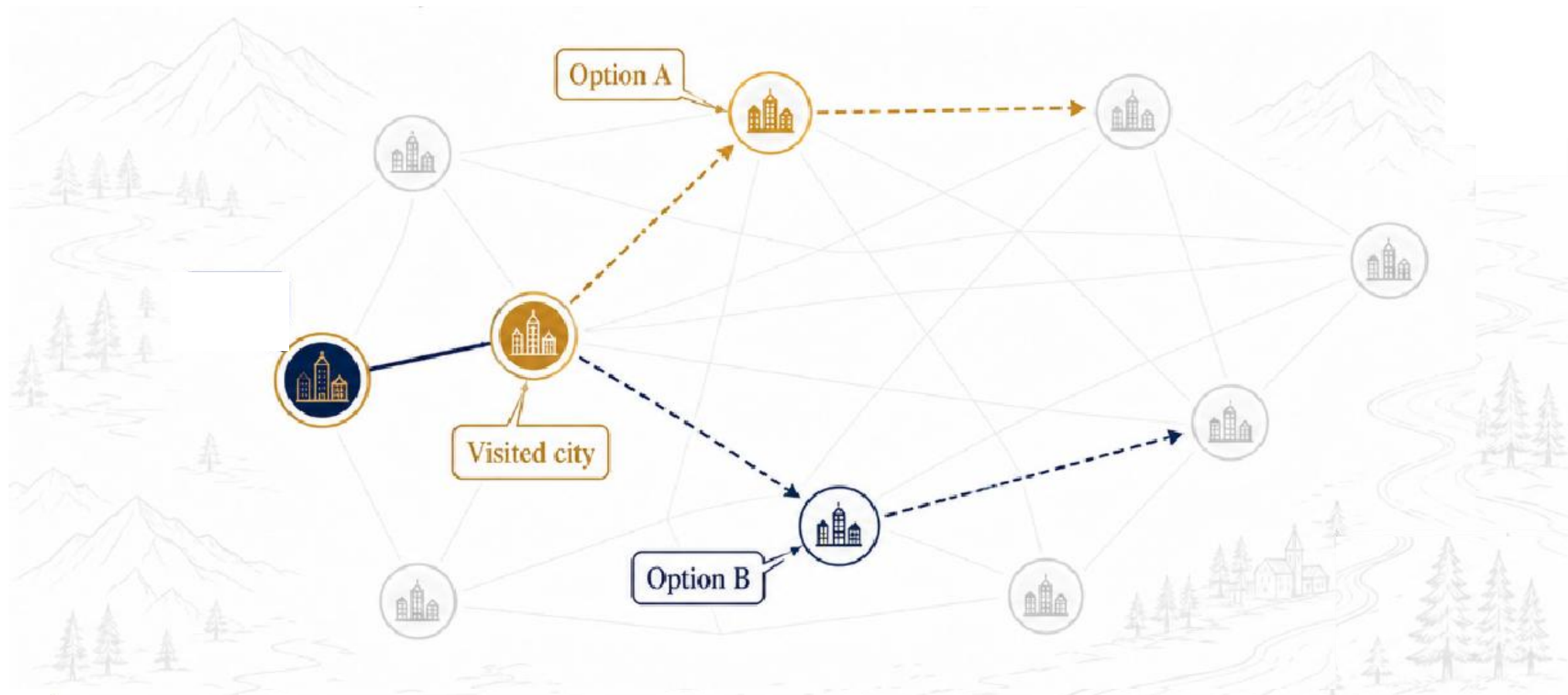




Introduction to Optimization and Metaheuristics



- Suppose you are planning a delivery route between 10 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?

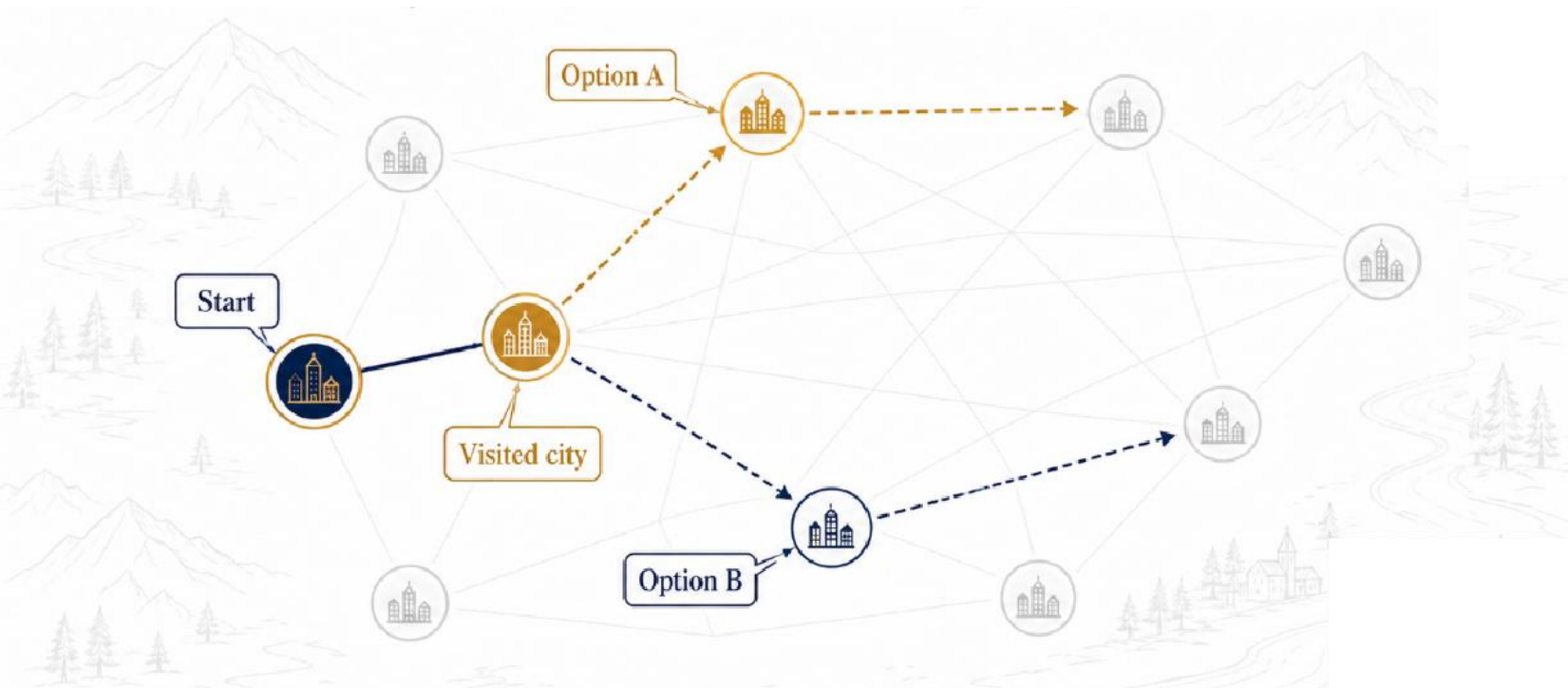




Introduction to Optimization and Metaheuristics



- Suppose you are planning a delivery route between 10 cities. You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?



Combinatorial Growth



*Each step multiplies
the number of possibilities.*

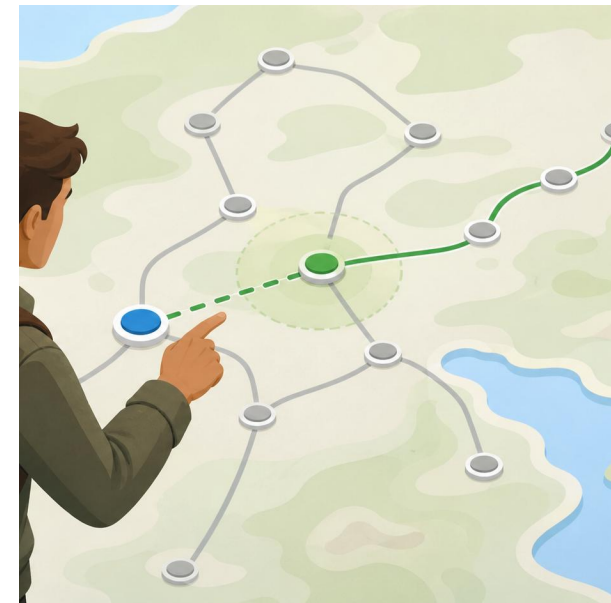


- Suppose you are planning a delivery route between 25 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?





- Suppose you are planning a delivery route between 10 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?



- ✓ For only 10 cities, the number of possible routes is extremely large.
- ✓ Exhaustive search quickly becomes impractical.
- ✓ This is where optimization and metaheuristic algorithms become useful.



Introduction to Optimization and Metaheuristics



- Suppose you are planning a delivery route between 10 cities.
You want to visit each city **exactly once** and return to the starting city.
- How many possible routes should we check to guarantee the best one?

📍 Example Tour (10 Cities)



As the number of cities increases, exhaustive search quickly becomes impractical.





How can we find high-quality solutions when exact search is too expensive?





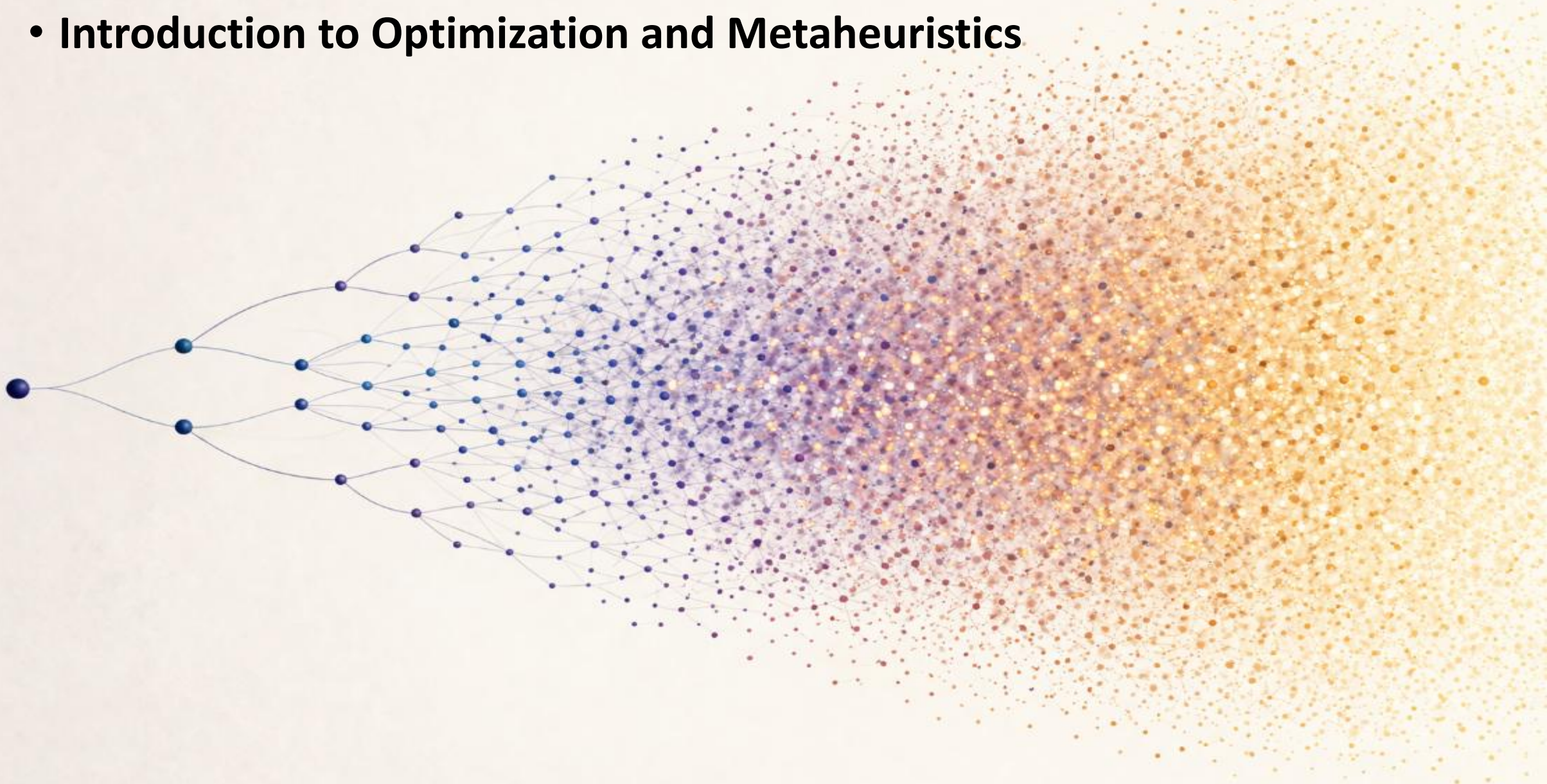
How can we find high-quality solutions when exact search is too expensive?

**Optimization gives us the language to define the problem.
Metaheuristics give us the intelligence to search.**





- **Introduction to Optimization and Metaheuristics**





Many real-world decisions are optimization problems.

- ✓ Choose the best option among many possibilities.
- ✓ Trade off cost, time, quality, accuracy, and risk.
- ✓ Examples: routing, scheduling, model tuning, design.
- ✓ A good decision is defined by a clear objective.





Many real-world decisions are optimization problems.

- ❑ **Decision:** the order of visiting cities.
- ❑ **Objective:** minimize total travel distance.
- ❑ **Constraint:** visit each city exactly once.
- ❑ **Challenge:** the route count grows very fast.

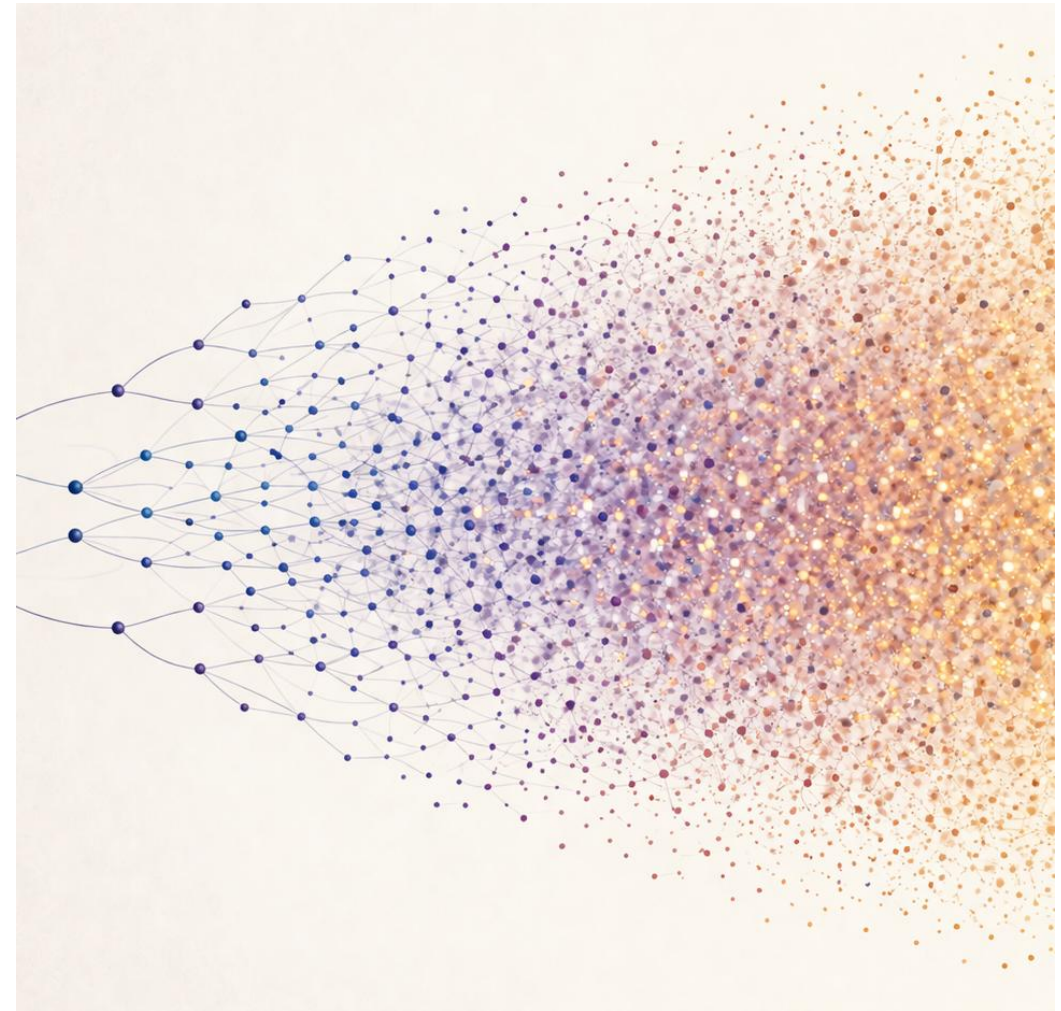




Trying all solutions quickly becomes impractical!

Cities	Possible tours
5	24
10	362,880
15	87,178,291,200
20	121,645,100,408,832,000

We need smarter search strategies than exhaustive enumeration.





What Is an Optimization Problem?



Find x that minimizes or maximizes $f(x)$, subject to $x \in S$.

Decision variables

What we can change: route order, model parameters, selected features.

Objective function

The quantity to optimize: cost, error, distance, profit.

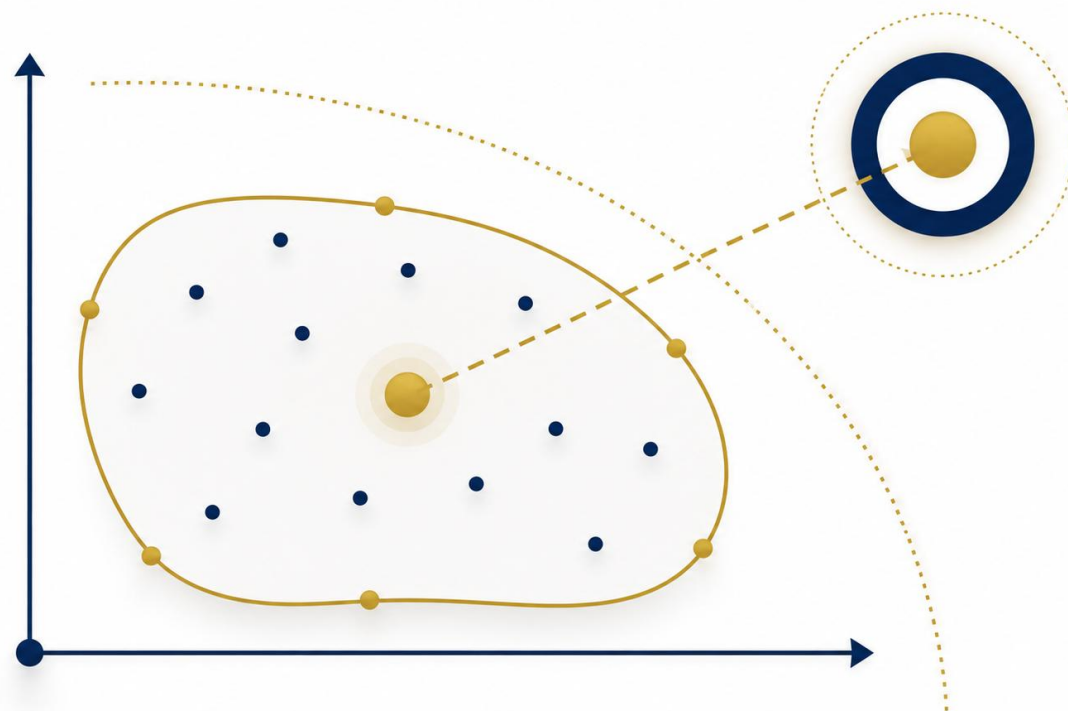
Constraints

Rules that solutions must satisfy: capacity, time, budget.

Search space

All possible candidate solutions allowed by the problem.

We need smarter search strategies than exhaustive enumeration.





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	?
Objective	What to optimize	?
Constraint	What must hold	?
Feasible solution	Valid candidate	?
Search space	All candidates	?





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	Order of cities
Objective	What to optimize	?
Constraint	What must hold	?
Feasible solution	Valid candidate	?
Search space	All candidates	?





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	Order of cities
Objective	What to optimize	Total route length
Constraint	What must hold	?
Feasible solution	Valid candidate	?
Search space	All candidates	?





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	Order of cities
Objective	What to optimize	Total route length
Constraint	What must hold	Each city visited once
Feasible solution	Valid candidate	?
Search space	All candidates	?





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	Order of cities
Objective	What to optimize	Total route length
Constraint	What must hold	Each city visited once
Feasible solution	Valid candidate	A complete tour
Search space	All candidates	?





Use one running example to avoid confusion: the Traveling Salesman Problem (TSP)

Component	Meaning	TSP example
Decision variables	What can change	Order of cities
Objective	What to optimize	Total route length
Constraint	What must hold	Each city visited once
Feasible solution	Valid candidate	A complete tour
Search space	All candidates	All possible tours

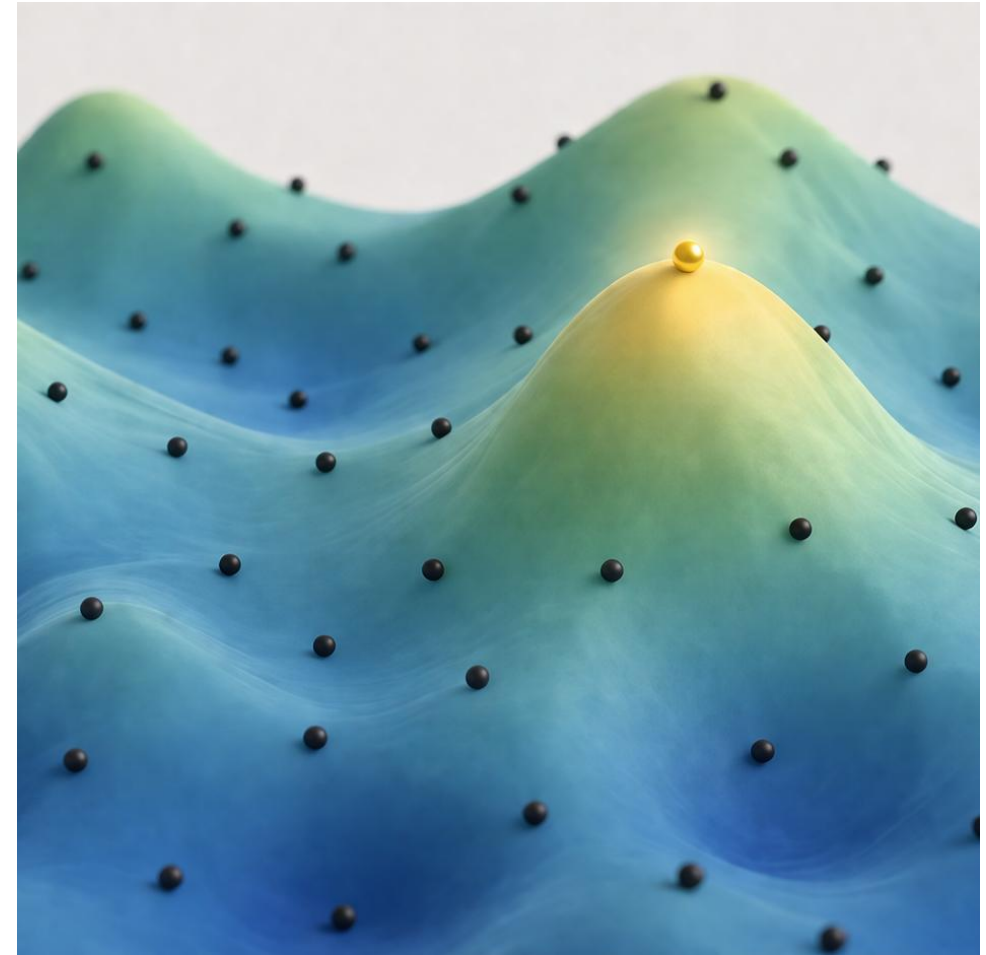
Optimization is not only about finding a solution; it is about finding the best solution according to a defined objective.



The direction of “better” depends on the objective!

- ❑ **Minimize:** cost, distance, error, energy, risk.
- ❑ **Maximize:** profit, accuracy, reliability, utilization.

- ❖ Many maximization problems can be converted to minimization.
- ❖ Always define what “better” means clearly.





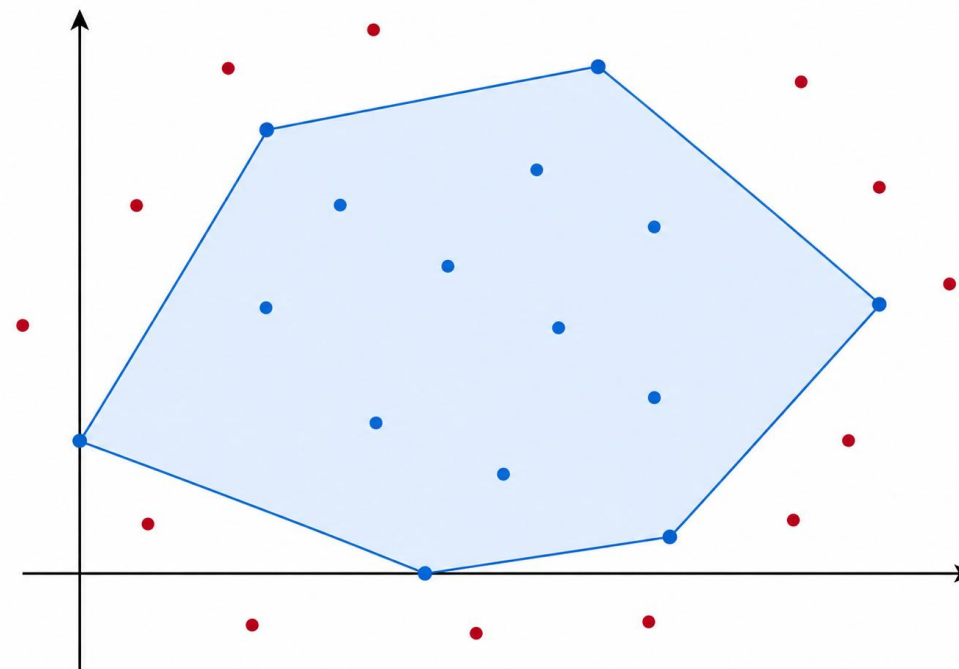
Different variable types often suggest different algorithms!

Discrete optimization

Variables are binary, integer, or permutations. Examples: TSP, knapsack, timetabling. Typical methods: GA, ACO, Tabu Search.

Continuous optimization

Variables take real values. Examples: engineering parameters, neural network weights, control tuning. Typical methods: PSO, DE, ES.

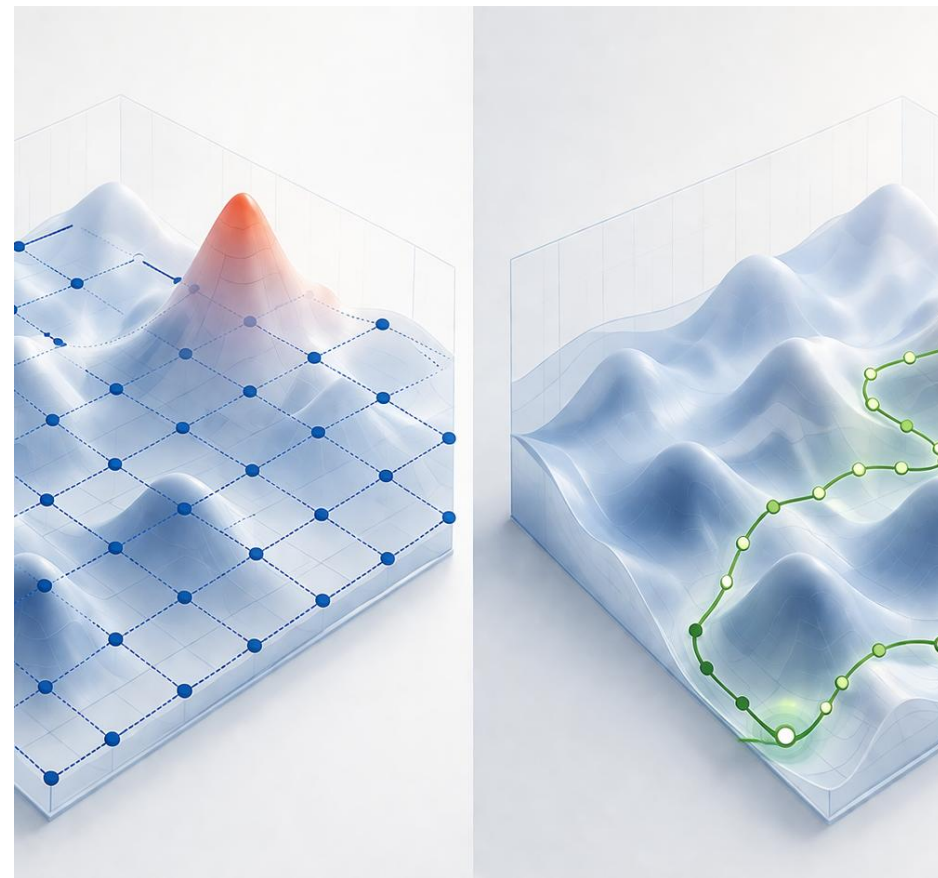




There is a practical trade-off between guarantee and speed!

Method type	Guarantee	Speed	Best suited for
Exact methods	?	?	?
Heuristics	?	?	?
Metaheuristics	?	?	?

A very good solution today is often more valuable than a perfect solution too late.

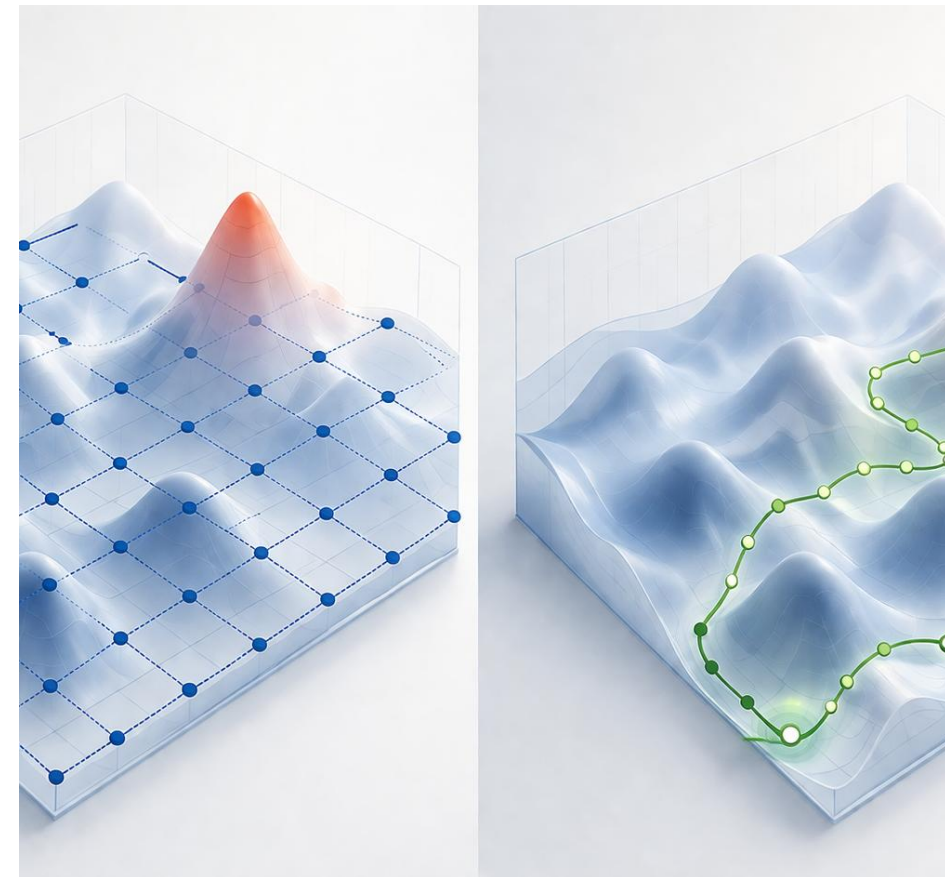




There is a practical trade-off between guarantee and speed!

Method type	Guarantee	Speed	Best suited for
Exact methods	?	?	Small / structured problems
Heuristics	?	Fast	Problem-specific solutions
Metaheuristics	?	?	Complex large-scale problems

A very good solution today is often more valuable than a perfect solution too late.

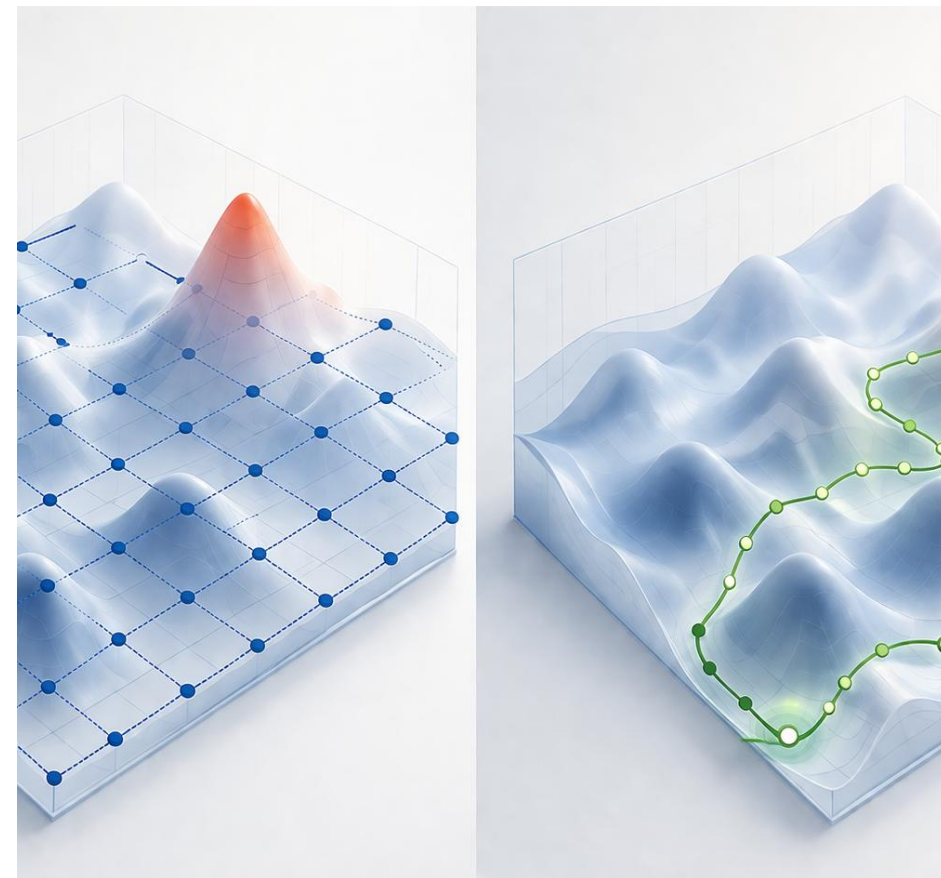




There is a practical trade-off between guarantee and speed!

Method type	Guarantee	Speed	Best suited for
Exact methods	?	?	Small / structured problems
Heuristics	No guarantee	Fast	Problem-specific solutions
Metaheuristics	?	?	Complex large-scale problems

A very good solution today is often more valuable than a perfect solution too late.

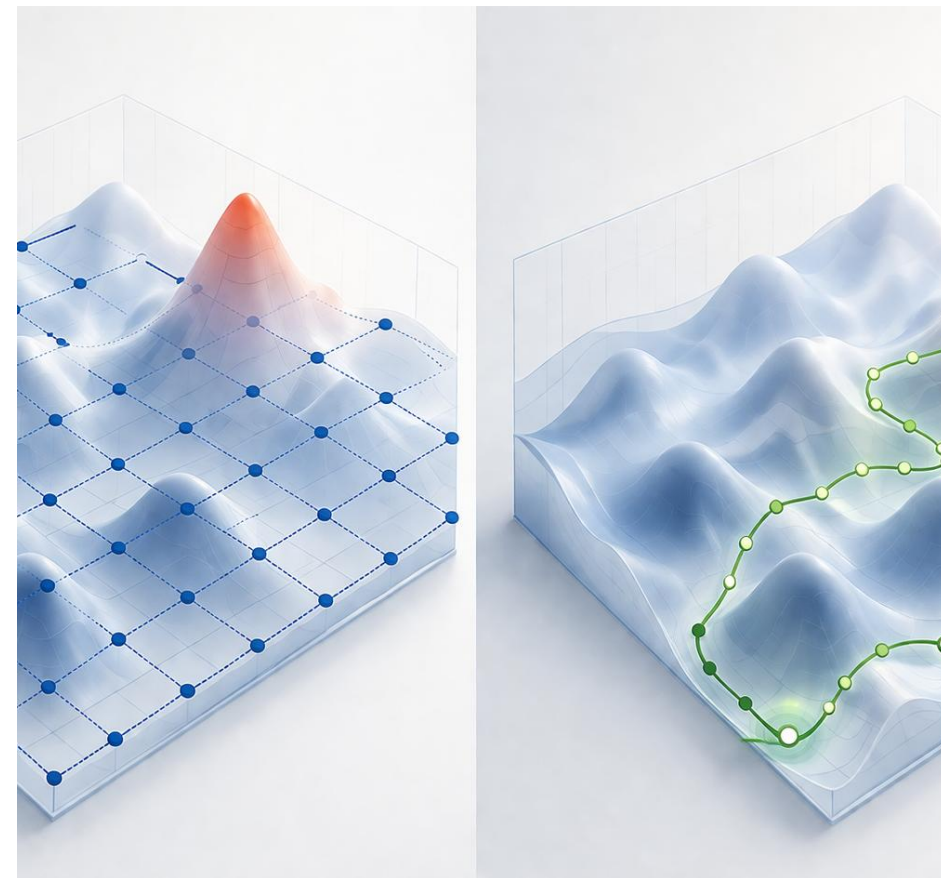




There is a practical trade-off between guarantee and speed!

Method type	Guarantee	Speed	Best suited for
Exact methods	Global optimum	Often slow	Small / structured problems
Heuristics	No guarantee	Fast	Problem-specific solutions
Metaheuristics	No guarantee	Flexible	Complex large-scale problems

A very good solution today is often more valuable than a perfect solution too late.







What did the quiz ask you to recognize?



- The same pattern appears in routing, machine learning, knapsack, scheduling, and facility location.

Decision variables

What can we change?

1

Objective function

What does “better” mean?

2

Constraints

What rules must hold?

3

Feasible solution

What is a valid candidate?

4

Search space

Where do all candidates live?

5

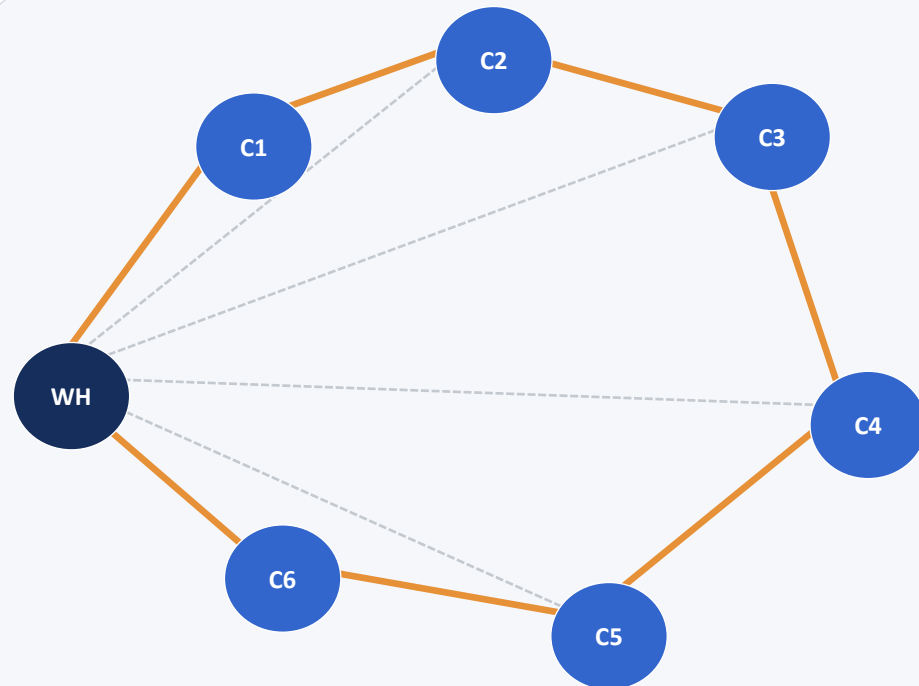
Ask: “For each quiz question, which component was easiest to identify? Which one was confusing?”



Quiz: Delivery Route Optimization (TSP-like example)



- Scenario: A truck starts from the warehouse, visits each customer exactly once, and returns to the warehouse. We want the shortest route.



Visual idea: each route is one candidate solution.

Decision variables: the order of visiting customers / cities

Objective: minimize total travel distance

Constraints: visit each customer exactly once; start and end at the warehouse

Feasible solution: a complete valid tour

Search space: all possible valid tours

Problem type: discrete, combinatorial, minimization



Quiz: Machine Learning Hyperparameter Tuning



- Scenario: Tune model hyperparameters to get the lowest validation error.
Example variables: learning rate, number of hidden layers, and batch size.



Visual idea: the knobs are the decision variables; the output score is the objective value.

Decision variables: hyperparameters such as learning rate, number of layers, batch size, etc.

Objective: minimize validation error (or maximize validation accuracy)

Constraints: allowed ranges for each hyperparameter

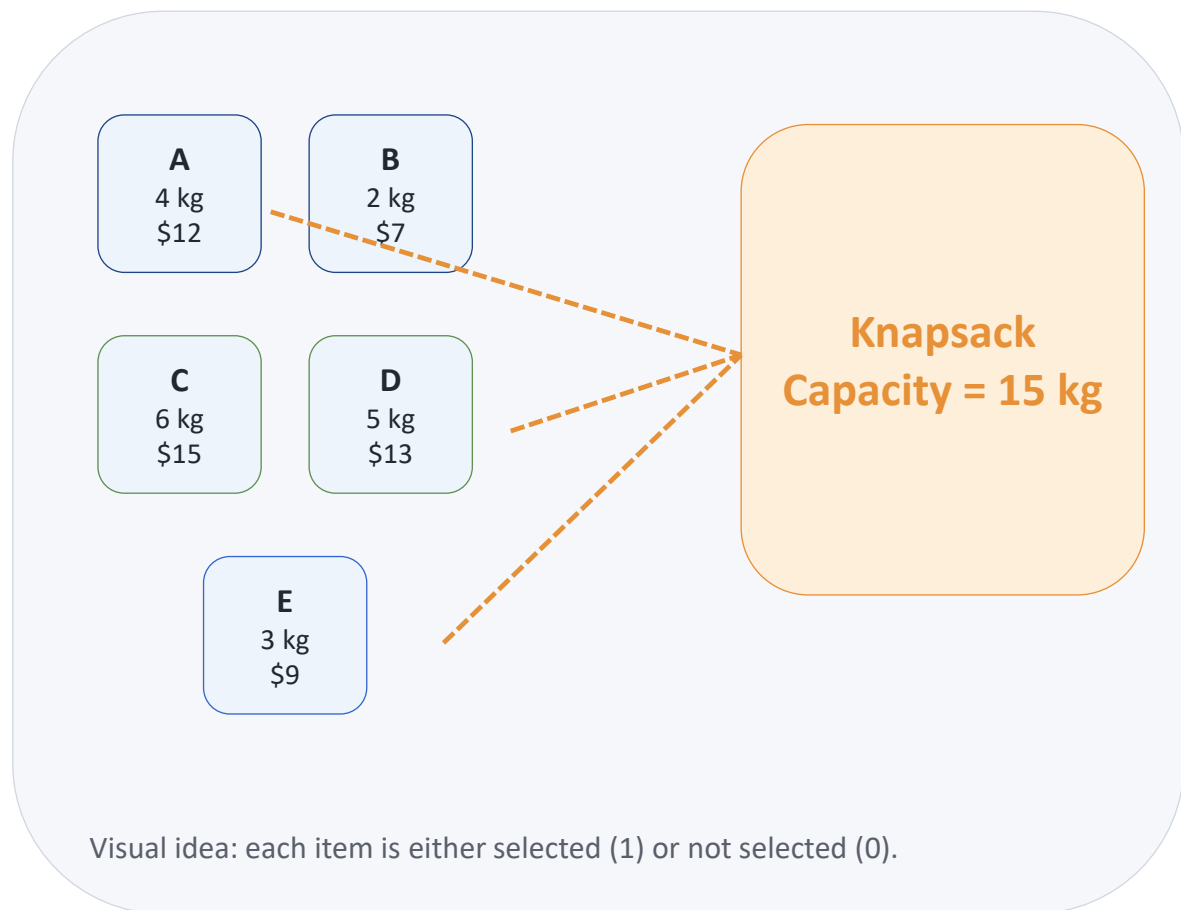
Feasible solution: any valid hyperparameter setting within the allowed ranges

Search space: all permitted combinations of hyperparameter values

Problem type: often mixed (continuous + discrete)



- Scenario: Select a subset of items to maximize total value, while the total weight stays within the bag capacity.



Decision variables: for each item, choose it (1) or not (0)

Objective: maximize total value / profit

Constraint: total weight must not exceed the capacity

Feasible solution: any subset of items within the weight limit

Search space: all possible subsets of items

Problem type: discrete, combinatorial, maximization

- Scenario: Assign exams to time slots so that no student has overlapping exams, while using as few exam days as possible.

Time	Day 1	Day 2	Day 3
9:00	Math	Physics	Programming
13:00	Economics	Biology	AI
16:00		History	

No overlaps

Visual idea: each placement chooses when an exam happens.

Decision variables: assign each exam to a day / time slot

Objective: minimize the number of exam days (or other scheduling cost)

Constraints: students cannot have overlapping exams; room/time limits may also exist

Feasible solution: a schedule with no rule violations

Search space: all possible assignments of exams to slots

Problem type: discrete optimization / combinatorial scheduling



Problem: Facility Location



- Scenario: Choose a limited number of warehouse locations to minimize customer distance, while staying within a construction budget.

Time	Day 1	Day 2	Day 3
9:00	Math	Physics	Programming
13:00	Economics	Biology	AI
16:00		History	

No overlaps

Visual idea: each placement chooses when an exam happens.

Decision variables: which candidate sites are selected

Objective: minimize average / total delivery distance to customers

Constraints: choose only a fixed number of facilities and keep total cost within budget

Feasible solution: a set of selected sites that satisfies the budget and count limits

Search space: all allowed combinations of candidate locations

Problem type: discrete combinatorial optimization



Compare the Problems



These examples differ in context, but their optimization structure is the same.

Problem	Decision variables	Objective	Main constraint	Type
Delivery route	Order of visits	Minimize distance	Visit each city once	Discrete / min
Hyperparameter tuning	Model settings	Minimize error	Value ranges	Mixed
Knapsack	Select items or not	Maximize value	Weight capacity	Discrete / max
Exam scheduling	Assign exams to slots	Minimize days / cost	No overlap conflicts	Discrete / min
Facility location	Choose sites	Minimize customer distance	Budget / number of sites	Discrete / min

Same logic: variables + objective + constraints

Feasible $\neq$ optimal

Different problems suggest different algorithms



- No matter what the application is, these 5 questions reveal the optimization structure:

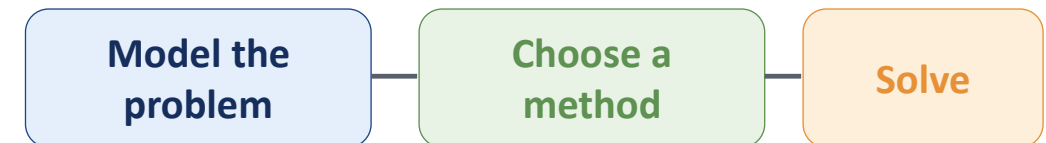
1. What can we change?

2. What are we trying to minimize or maximize?

3. What rules must hold?

4. What does a valid solution look like?

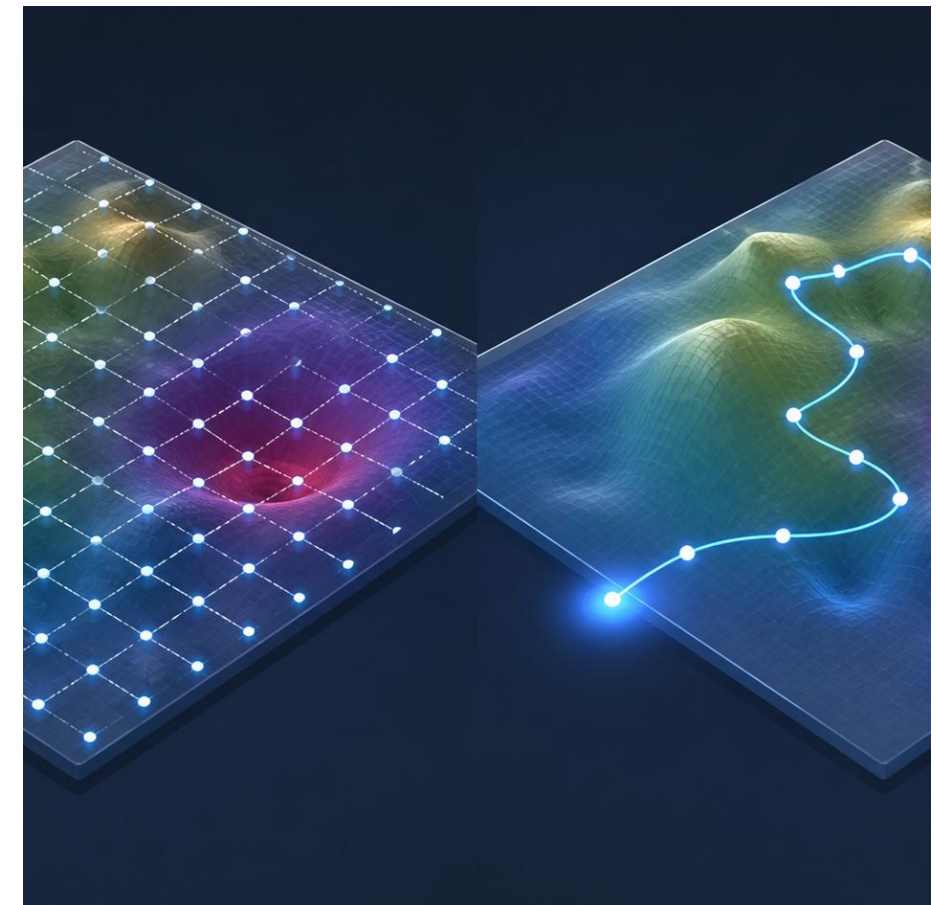
5. How large is the search space?





A fast rule of thumb designed for a specific problem!

- ✓ Usually problem-specific.
- ✓ Often simple and efficient.
- ✓ Does not guarantee the global optimum.
 - Example: in routing, always visit the nearest unvisited city.



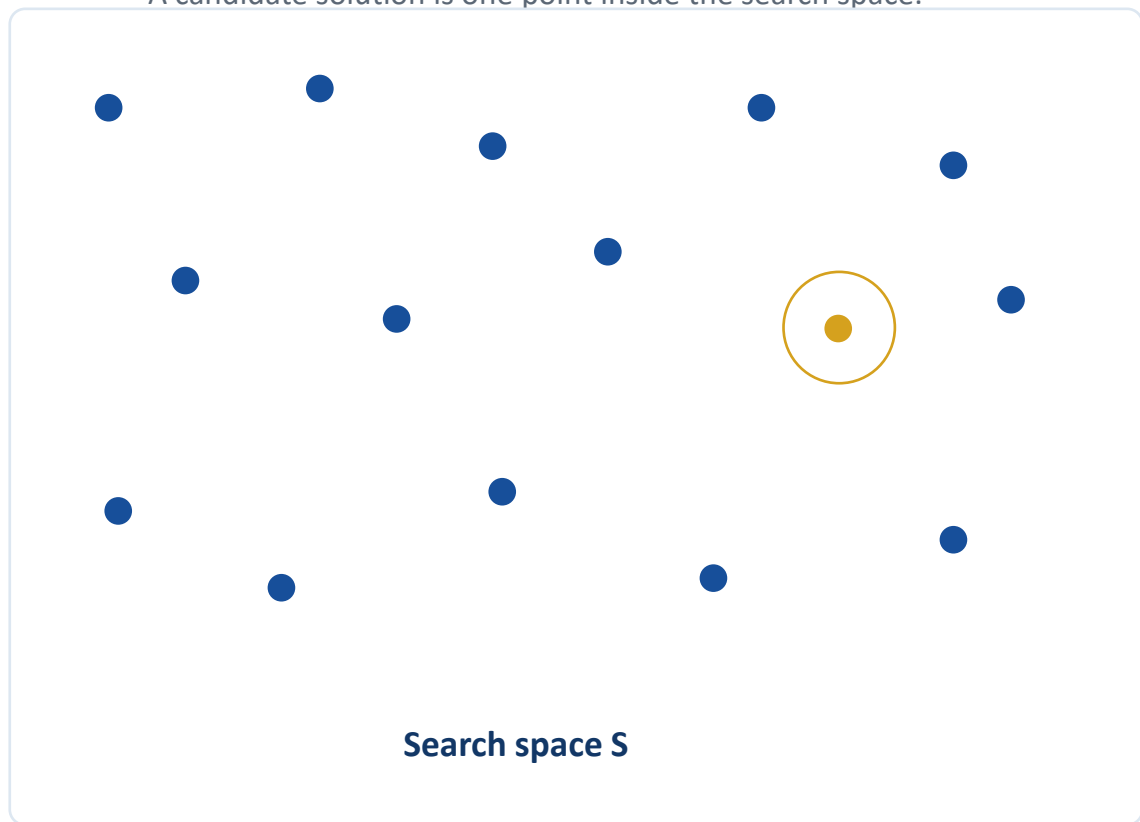


What Is a Heuristic?



Optimization is movement through alternatives

A candidate solution is one point inside the search space.



Search space

All possible solutions.

Candidate

One possible solution.

Objective value

Quality score of a candidate.

Process

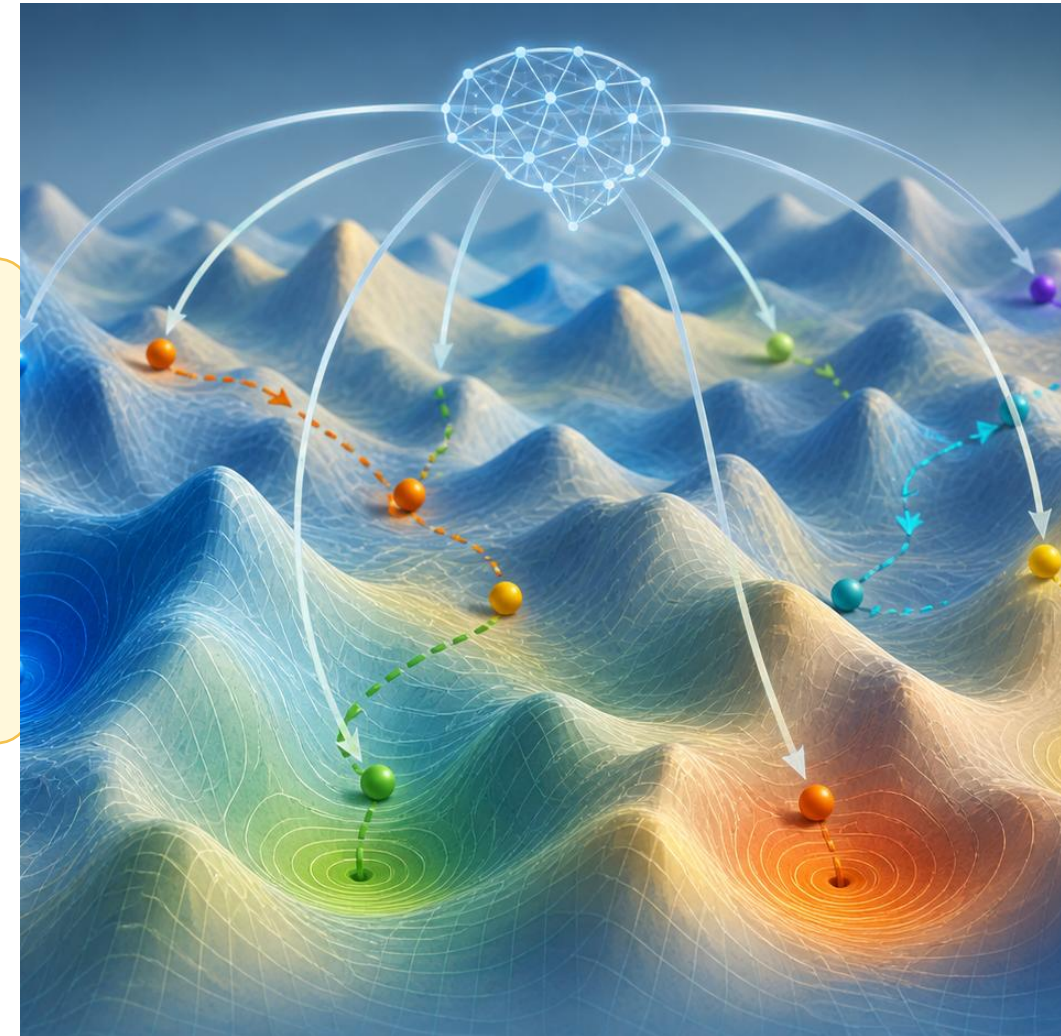
Move toward better candidates.

Every answer described a different part of search space.



Optimization is movement through a space of alternatives!

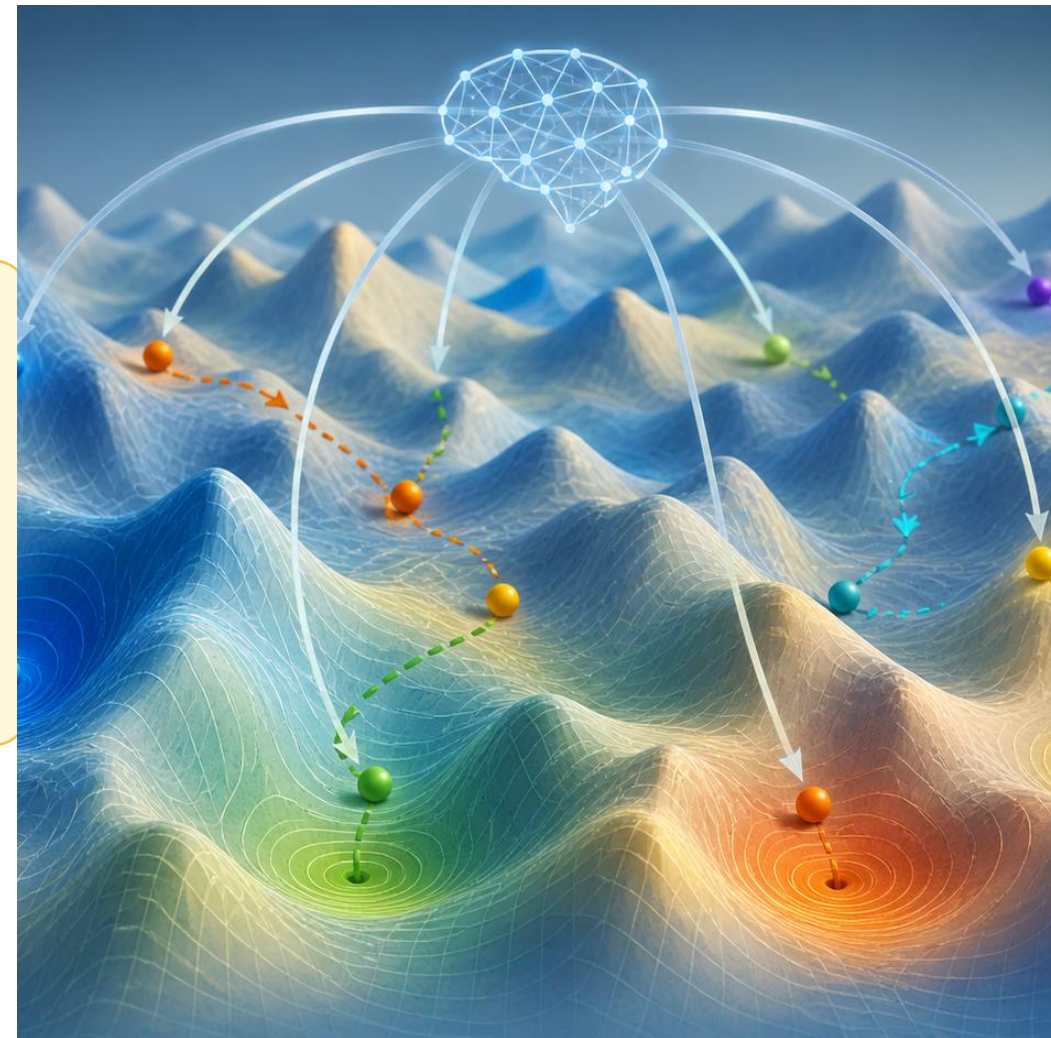
- ✓ **Search space:** all possible solutions.
- ✓ **Candidate solution:** one point inside that space.
- ✓ **Objective value:** quality of a candidate.
- ✓ **Optimization process:** move toward better candidates.





The objective function turns a solution into a number!

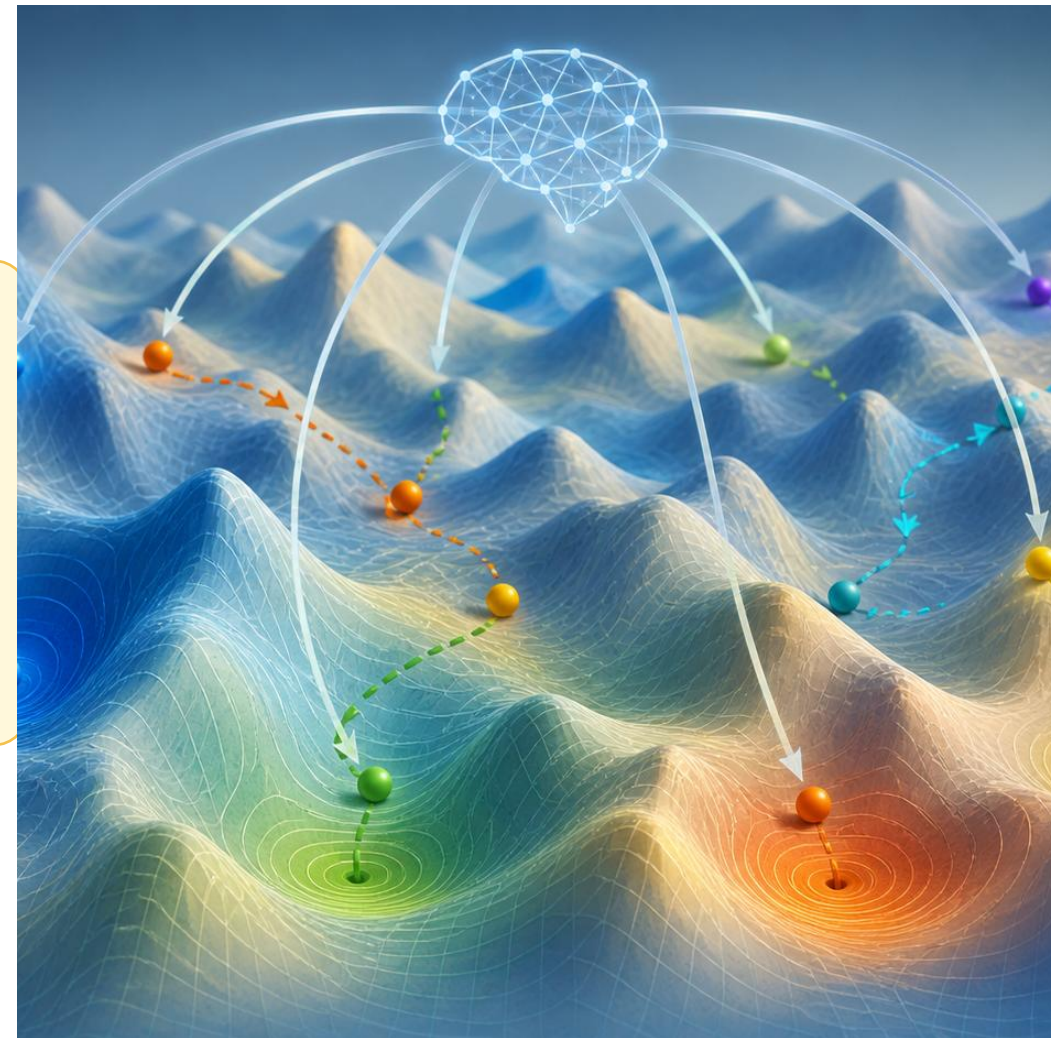
- ☐ TSP: ?
- ☐ Machine learning: ?
- ☐ Knapsack: ?
- ☐ ...





The objective function turns a solution into a number!

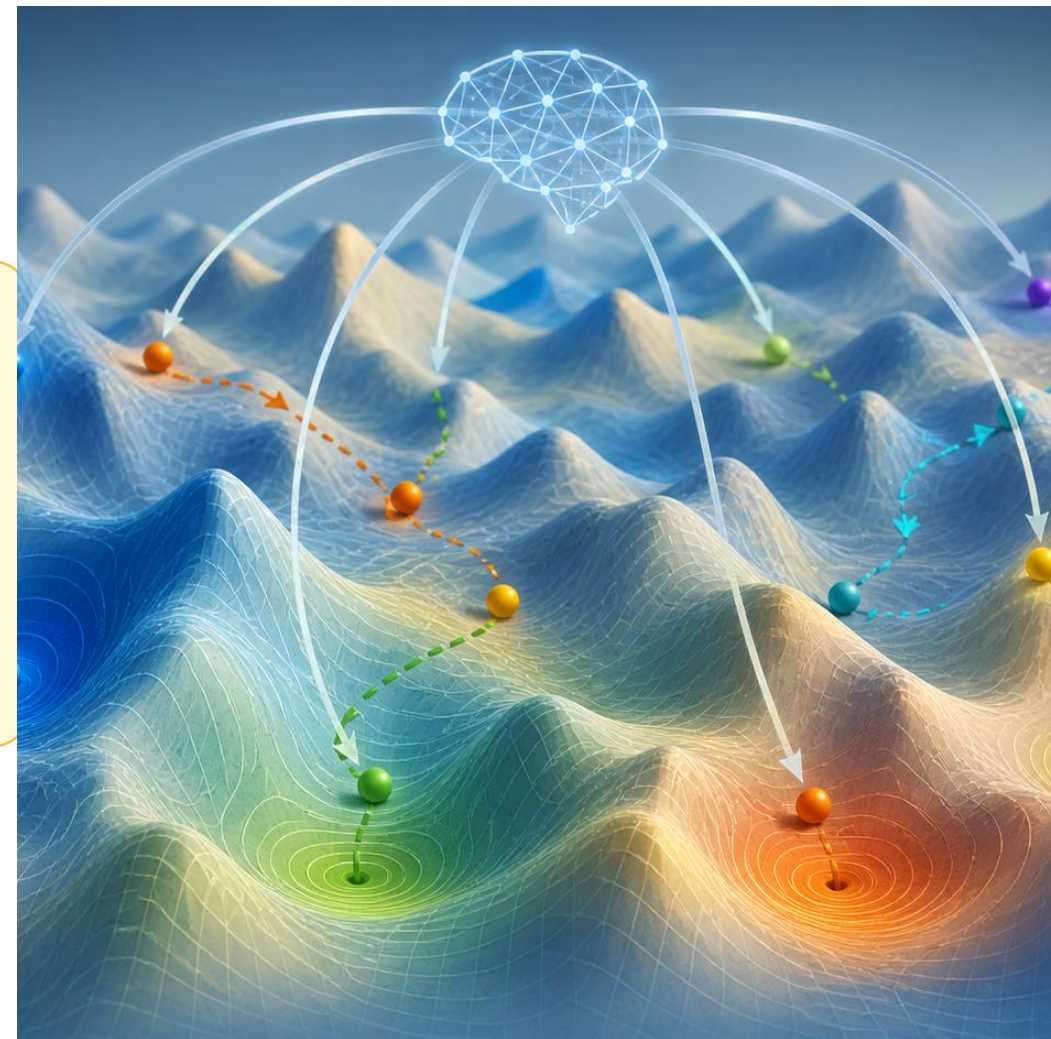
- ☐ **TSP:** total distance of the route.
- ☐ **Machine learning:** ?
- ☐ **Knapsack:** ?
- ☐ ...





The objective function turns a solution into a number!

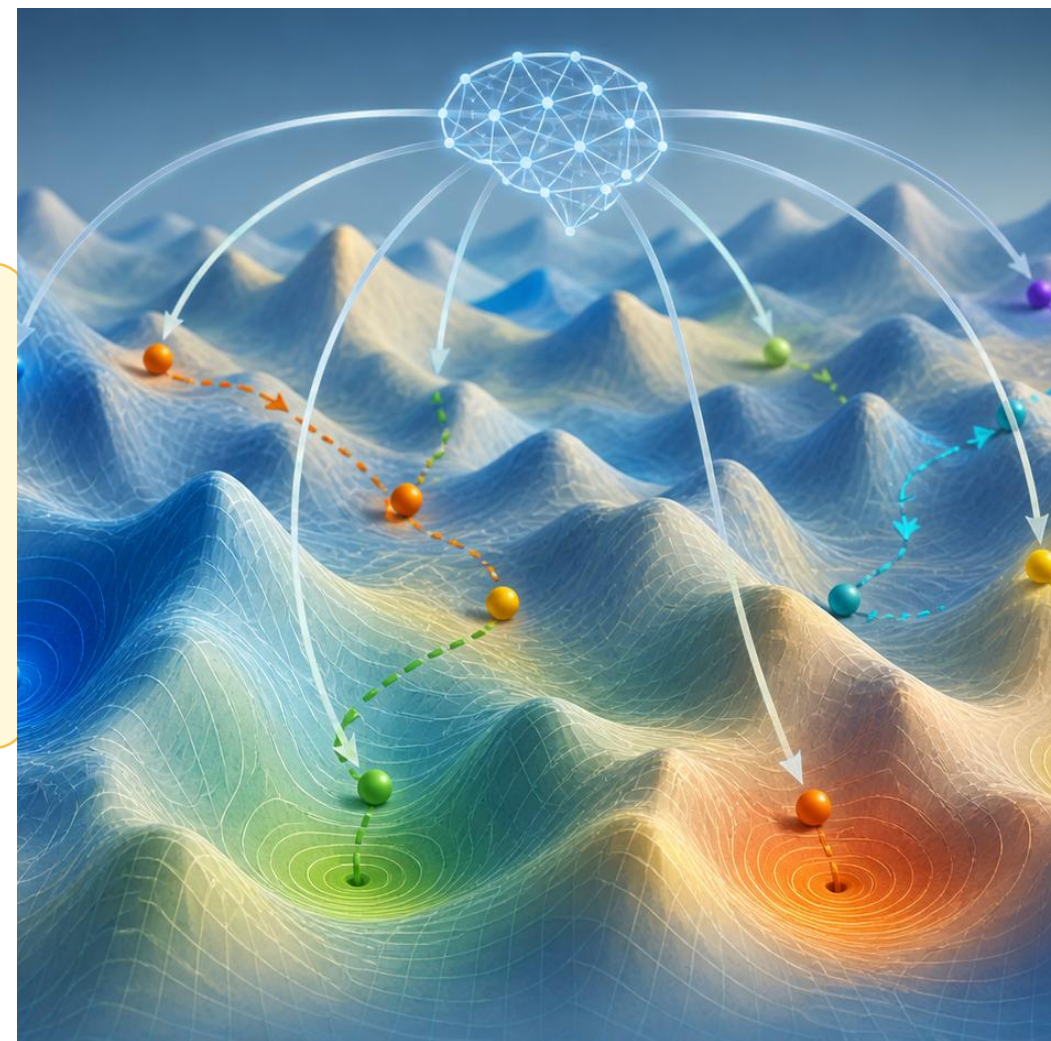
- ☐ **TSP:** total distance of the route
- ☐ **Machine learning:** validation error.
- ☐ **Knapsack:** ?
- ☐ ...





The objective function turns a solution into a number!

- ❑ **TSP:** total distance of the route.
- ❑ **Machine learning:** validation error.
- ❑ **Knapsack:** total profit under capacity.
- ❑ A poor objective definition leads to poor optimization.





Constraints define the rules of the game!





Constraints define the rules of the game!



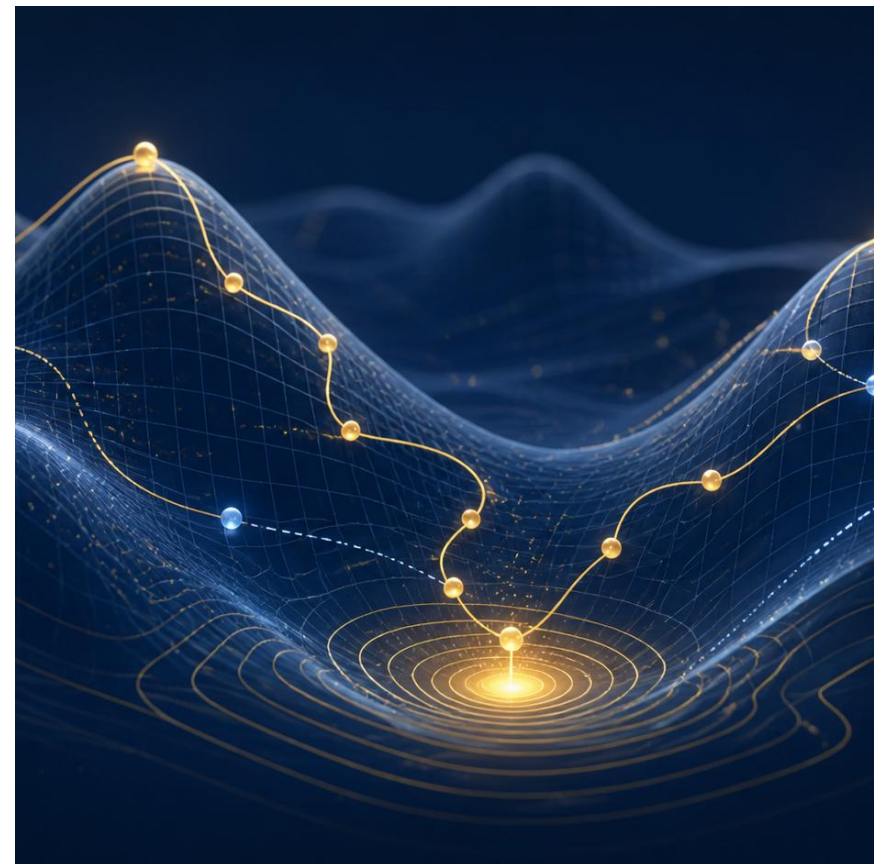
- ☐ A feasible solution satisfies all constraints.
- ☐ An infeasible solution violates at least one rule.
- ☐ Common handling methods: penalty, repair, decoder, rejection.
- ☐ Ask students: “Why is this solution invalid?”



One of the central challenges in metaheuristics!

- ❑ **Local optimum:** better than nearby solutions.
- ❑ **Global optimum:** best in the entire search space.

- Algorithms may become trapped locally.
- Escaping local optima is a key design goal.



A good metaheuristic must discover and refine!

Exploration

Search new and unknown regions.

Benefit: diversity.

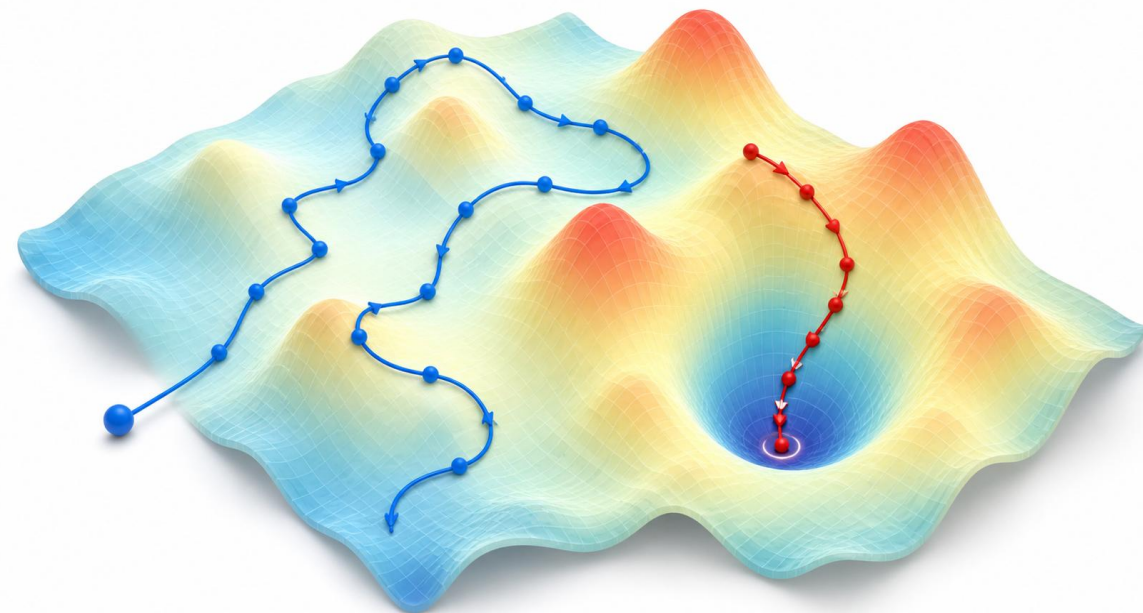
Risk: too random.

Exploitation

Improve promising regions.

Benefit: precision.

Risk: local traps.



A good metaheuristic must discover and refine!

Exploration

Search new and unknown regions.

Benefit: diversity.

Risk: too random.

Exploitation

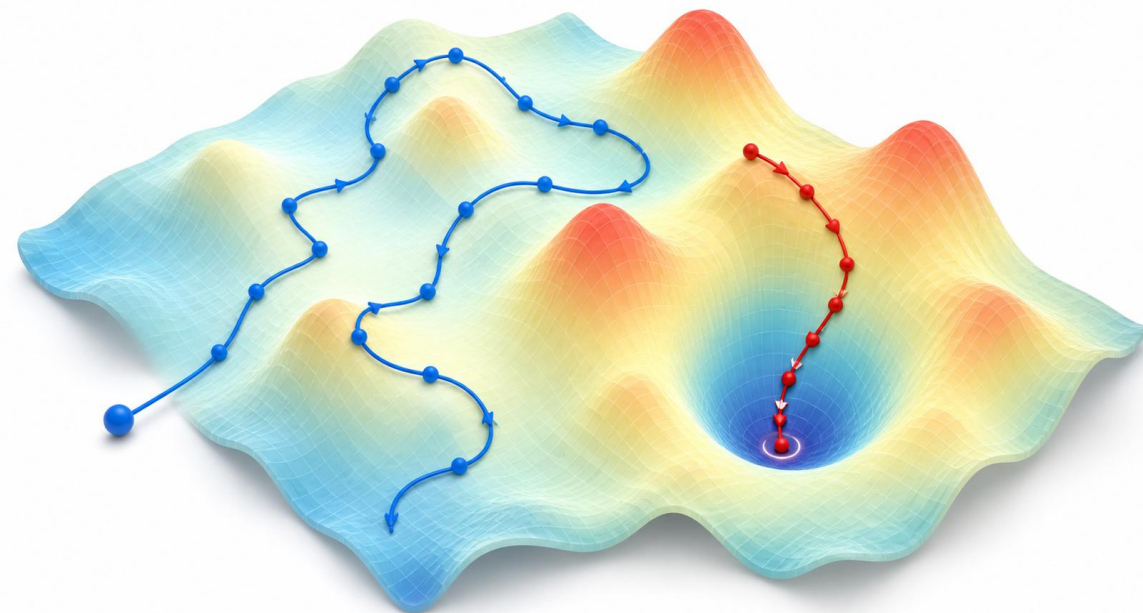
Improve promising regions.

Benefit: precision.

Risk: local traps.

Exploration helps us discover!

Exploitation helps us refine!





Most metaheuristics follow a simple iterative pattern.

```
Initialize solution or population
```

```
While stopping condition is not met:
```

```
    Generate new candidates
```

```
    Evaluate candidates
```

```
    Update current / best solutions
```

```
Return best solution found
```





Metaheuristics connect theory with real-world decision making!

Engineering

Design optimization, structural tuning, control parameters.

Artificial Intelligence

Feature selection, hyperparameter tuning, architecture search.

Operations Research

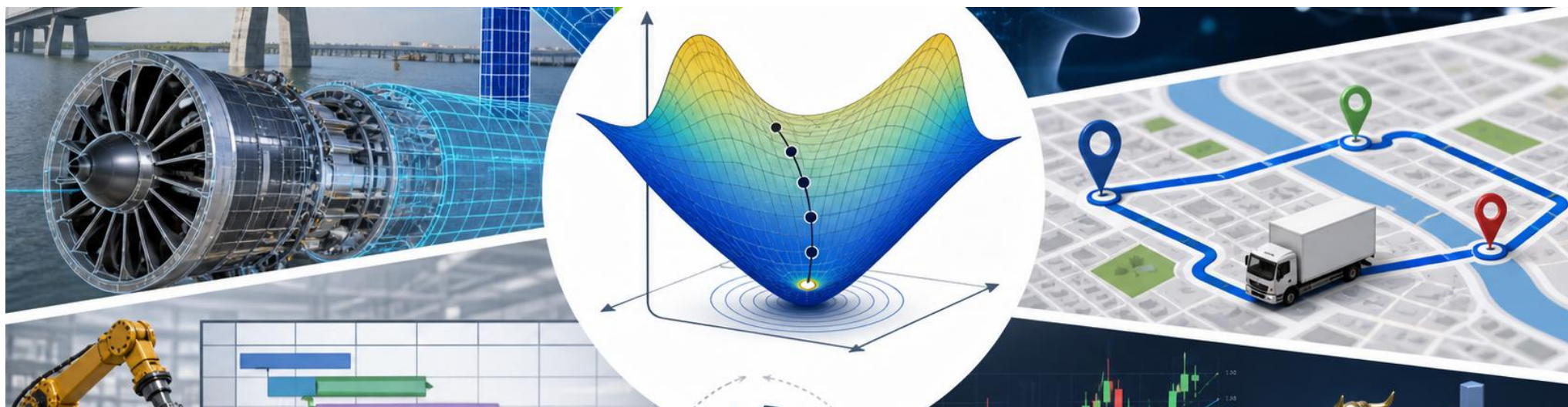
Scheduling, routing, facility location, allocation.

Data Science

Clustering, dimensionality reduction, model selection.

Finance

Portfolio optimization, risk management, decision support.

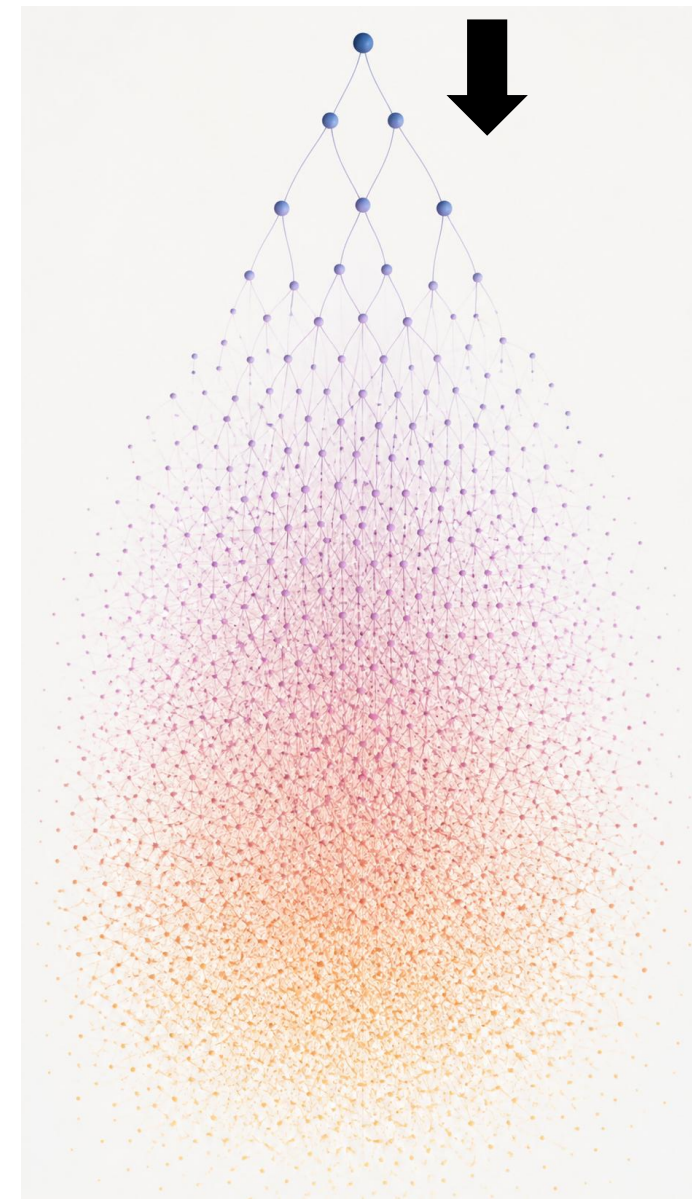




Metaheuristics connect theory with real-world decision making!

- ✓ **Optimization** is about the **best decision** under given conditions.
- ✓ **Search spaces** can become too large for exhaustive search.
- ✓ **Objective functions** evaluate solution quality.
- ✓ Constraints separate feasible from infeasible solutions.
- ✓ Metaheuristics trade guarantees for flexibility, speed, and practical usefulness.

Next: single-solution metaheuristics — Random Search and Hill Climbing.





Next stop: Random Search and Hill Climbing



- How does one candidate solution move through the search space?

Random Search

Generate candidates randomly and keep the best one found. Simple baseline for comparison.

Hill Climbing

Move from the current solution to a better neighboring solution. Fast, but can get stuck.

Core question

How can we improve solutions while avoiding local traps?

End with: "In the next session, we will implement this movement idea step by step."



✓ Phase 2: Random Search & Hill Climbing

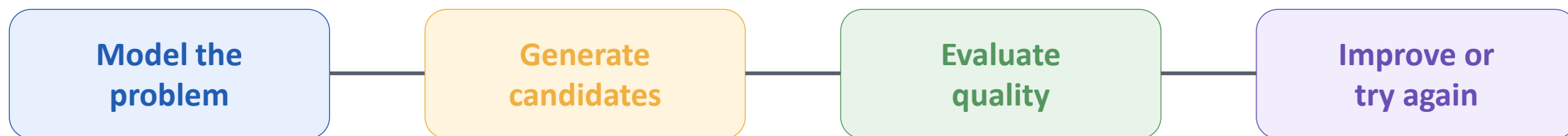


Delve more ... step by step!



From modeling the problem to searching for solutions

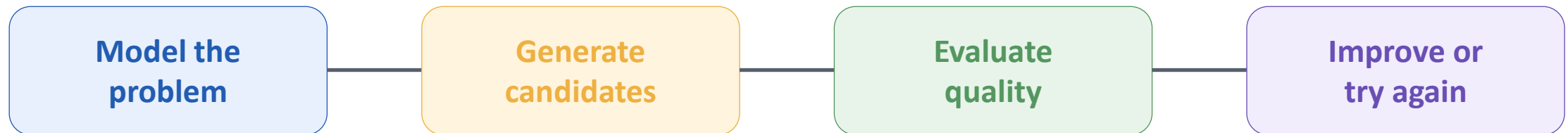
In the previous section, we learned how to identify decision variables, objective functions, constraints, feasible solutions, and search spaces.





From modeling the problem to searching for solutions

In the previous section, we learned how to identify decision variables, objective functions, constraints, feasible solutions, and search spaces.



Today we study two baseline search strategies that make this process concrete.

Two first algorithms

- **Random Search:** sample candidate solutions without memory
- **Hill Climbing:** move step by step to better neighbors



Why Start with These Algorithms?



- They are simple, but they reveal the main ideas behind many metaheuristics.

Random Search teaches us:

- How to sample the search space
- Why evaluation is the core operation
- How baseline performance is measured
- Why no memory can be inefficient

Hill Climbing teaches us:

- How to use neighborhood moves
- How local improvement works
- Why local optima are dangerous
- Why escaping mechanisms are needed

They are the baseline for understanding advanced metaheuristics.



- Try many candidate solutions randomly, evaluate them, and keep the best one found.



Each dot is a random candidate solution.

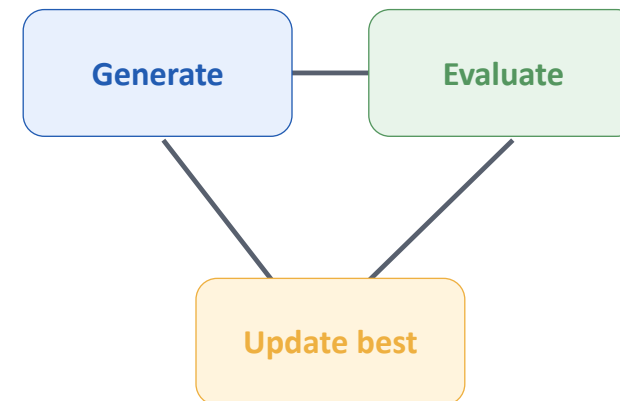
Main components

- **Representation:** how a solution is encoded
- **Random generator:** how candidates are produced
- **Objective function:** how quality is measured
- **Best-so-far memory:** the best solution found until now
- **Stopping condition:** time, iterations, or evaluation budget

Random Search

Initialize best_solution = None
Initialize best_value = worst possible value
Repeat until stopping condition:
 Generate a random candidate solution
 Evaluate candidate using $f(x)$
 If candidate is better than best_solution:
 Update best_solution and best_value
Return best_solution

Generate → Evaluate → Keep best



The algorithm has no sense of direction. It depends on enough sampling to discover good regions.



Strengths

- Very easy to implement
- Works with almost any representation
- Can sample many disconnected regions
- Good baseline for comparing smarter methods
- Parallelizable: candidates can be evaluated independently

Limitations

- Does not exploit good solutions
- May waste evaluations in poor regions
- Performance decreases in very large search spaces
- No learning from previous candidates
- Usually not enough for difficult problems



Strengths

- Very easy to implement
- Works with almost any representation
- Can sample many disconnected regions
- Good baseline for comparing smarter methods
- Parallelizable: candidates can be evaluated independently

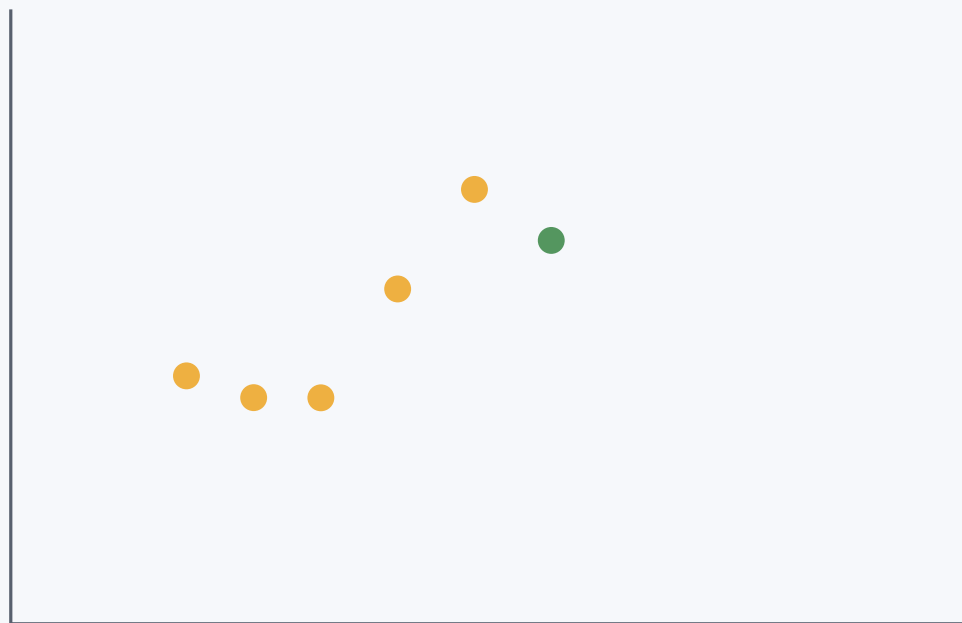
Limitations

- Does not exploit good solutions
- May waste evaluations in poor regions
- Performance decreases in very large search spaces
- No learning from previous candidates
- Usually not enough for difficult problems

Random Search is not “bad”; it is honest. It tells us what can be achieved without any intelligent guidance.
A good metaheuristic should beat this baseline.



- Start from one solution and repeatedly move to a better neighboring solution.



The path shows step-by-step local improvement.

Main components

- Current solution: where the search is now
- Neighborhood: solutions reachable by a small change
- Move operator: how neighbors are created
- Acceptance rule: accept the neighbor if it is better
- Stopping condition: no better neighbor or max iterations



- A neighborhood defines what “nearby solutions” mean for a specific problem.

Example: TSP swap move



Swap two cities to create a neighbor:



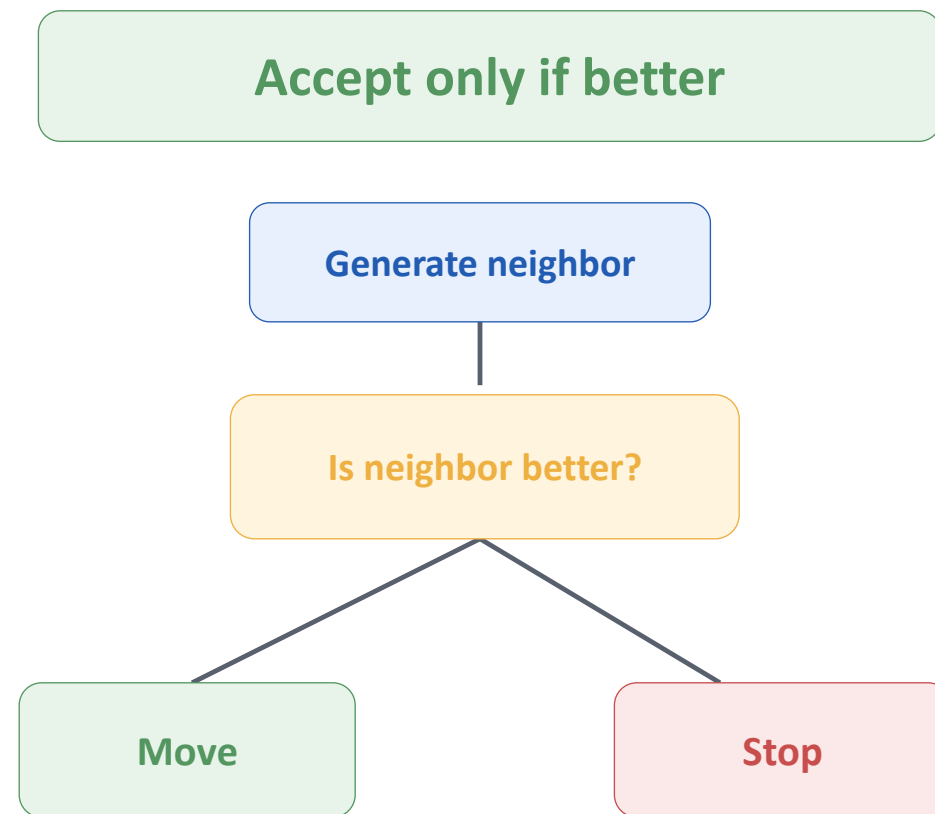
Why neighborhood matters

- It controls the steps the algorithm can take
- A small neighborhood is fast but may be limited
- A large neighborhood explores more but costs more
- The same algorithm can behave differently with different move operators

for knapsack, flip one item; for exam scheduling, move one exam to another slot; for hyperparameters, slightly change one setting.

Hill Climbing

Generate an initial solution x
Repeat until stopping condition:
 Generate neighbors of x
 Select the best improving neighbor x'
 If $f(x')$ is better than $f(x)$:
 Move to x'
 Else:
 Stop: x is locally optimal
Return x

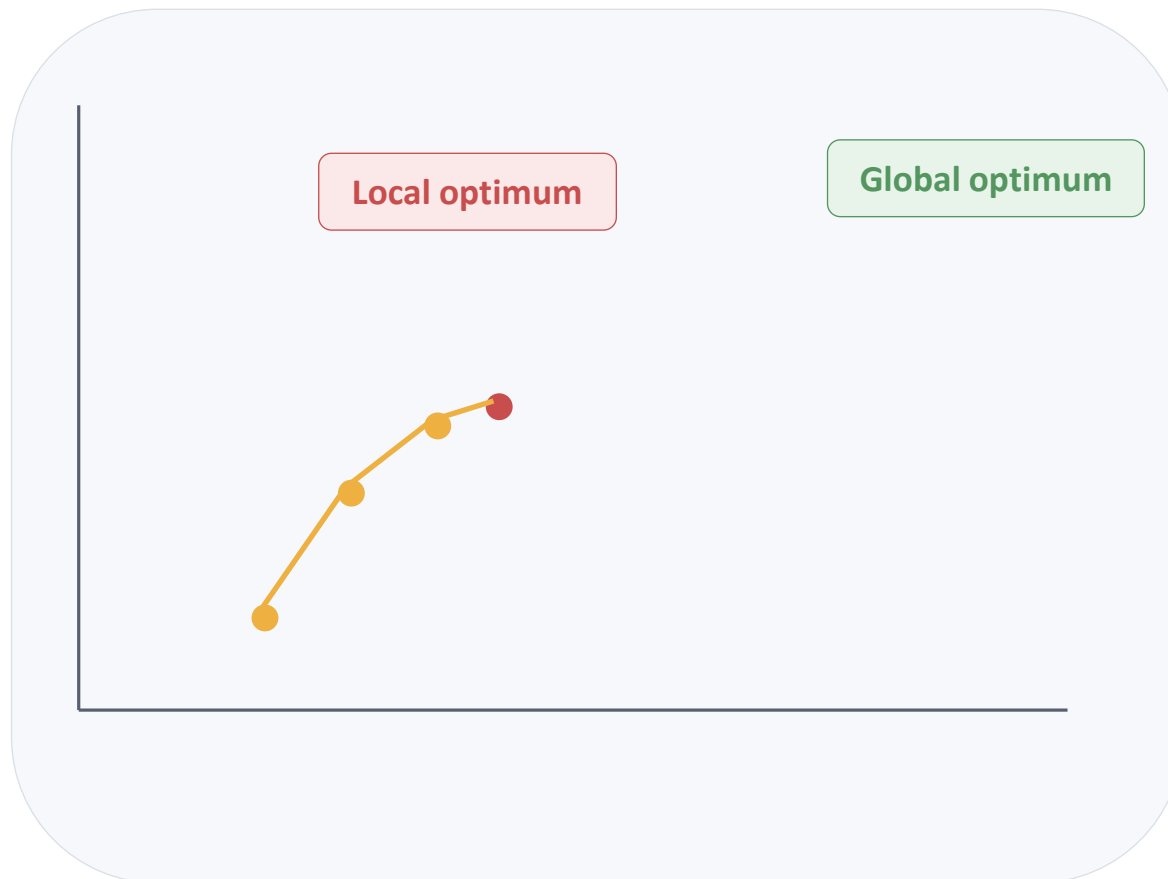


This is a greedy local search strategy.

Highlight the greedy acceptance rule: unlike random search, hill climbing learns locally, but it may become trapped.



- Hill Climbing can stop even when better solutions exist elsewhere.



Why it happens

- Hill Climbing compares only nearby solutions
- If all neighbors are worse, the algorithm stops
- The current solution may be locally best but globally poor
- This motivates advanced methods such as random restarts, simulated annealing, and tabu search

Use the landscape: the algorithm is not “wrong”; it is locally rational. The problem is that local information is incomplete.



- Before advanced metaheuristics, we can already improve basic Hill Climbing.

Steepest-Ascent

Evaluate many neighbors and choose the best improving one.

First-Choice

Scan neighbors and move to the first improving one.

Stochastic

Choose a random improving neighbor to add diversity.

Random Restart

Run Hill Climbing multiple times from different initial solutions.

These improvements introduce controlled randomness and better exploration.

This is a good place to foreshadow the next algorithms. Randomness and memory are two common ways to escape local traps.



Random Search vs. Hill Climbing



- Both are simple, but they represent two very different search behaviors.

Aspect	Random Search	Hill Climbing
Starting point	Many independent random candidates	One current solution
Guidance	No local guidance	Uses neighborhood improvement
Memory	Only best-so-far	Current solution + best-so-far
Strength	Simple, broad sampling	Fast local improvement
Weakness	Wastes evaluations	Can get stuck in local optima
Best role	Baseline / exploration	Local search / exploitation

Random Search explores broadly; Hill Climbing exploits locally.

Random Search is mostly exploration; Hill Climbing is mostly exploitation



- Use a small TSP, knapsack, or scheduling example
 - simulate both algorithms manually!

Activity steps

- Choose a small problem with 5–6 elements
- Define a solution representation
- Run 5 iterations of Random Search
- Run 5 moves of Hill Climbing
- Compare the best solutions and number of evaluations

Discussion questions

- Which method improved faster?
- Which method explored more alternatives?
- Where did Hill Climbing get stuck?
- How could we improve it?
- What role does randomness play?

Next: using randomness, memory, or probability to escape local optima.

This creates a smooth transition to simulated annealing, tabu search, genetic algorithms, or other metaheuristics.



Random Search

samples broadly and keeps the best candidate

Hill Climbing

improves one solution through local moves

Neighborhood

defines what a small change means

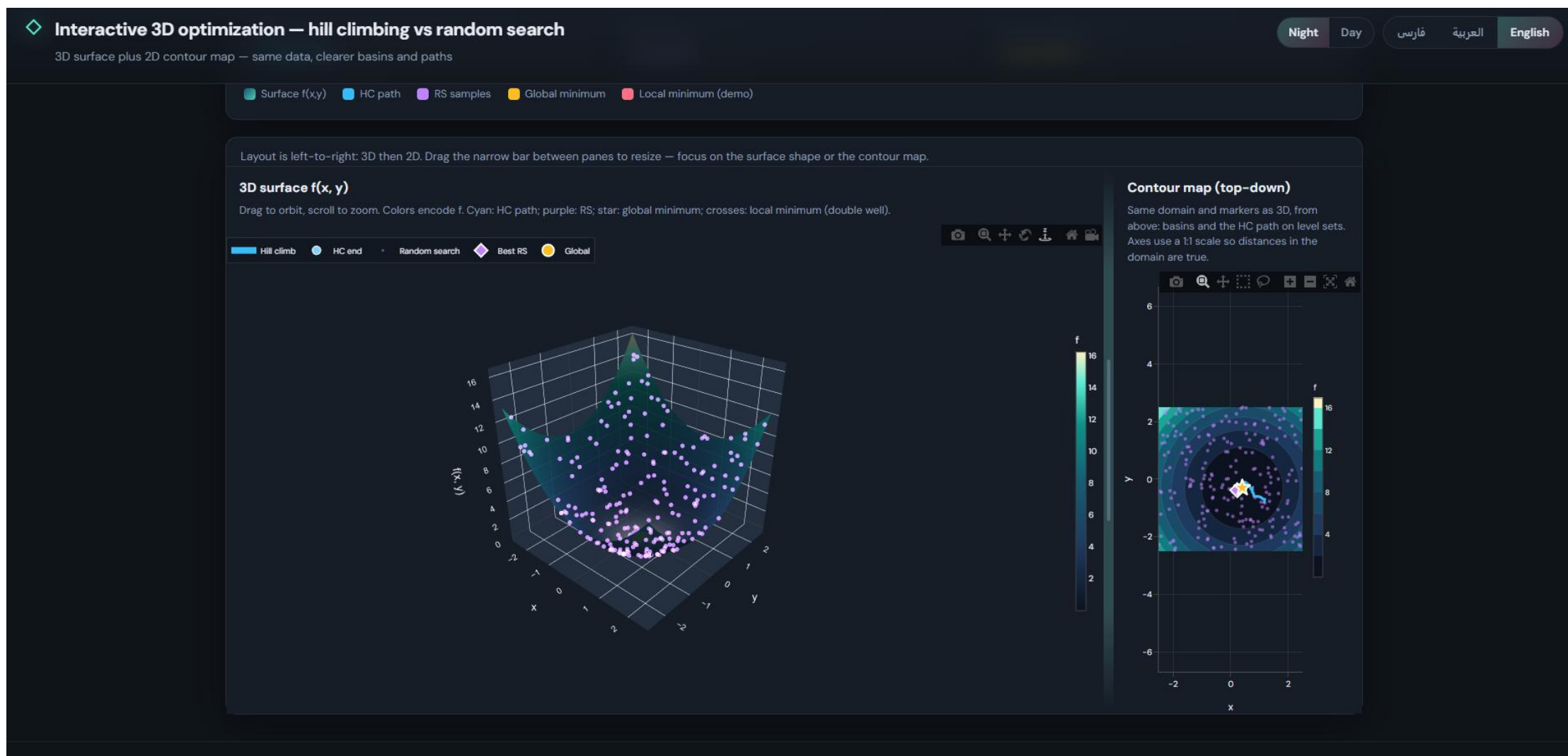
Local optimum

can trap greedy local search

Next step

add mechanisms to balance exploration and exploitation

Advanced metaheuristics are mostly smarter ways to generate, evaluate, accept, remember, and diversify solutions.



<https://khalooei.github.io/visualize-optimization/>

[Visualize Optimization](https://khalooei.github.io/visualize-optimization/)

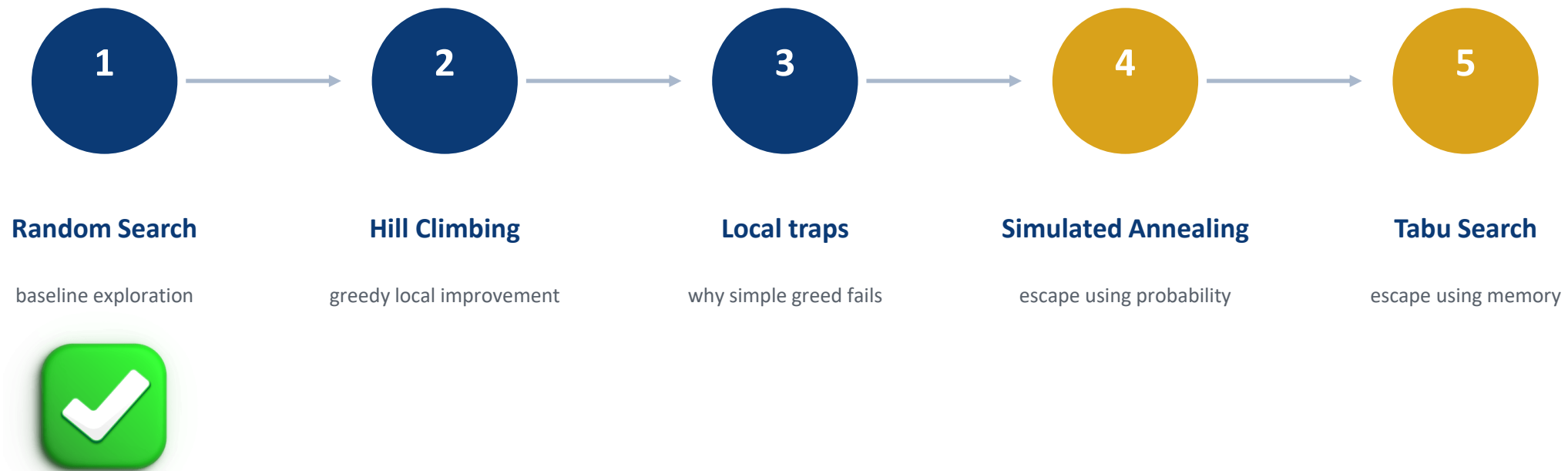


- We now finish the missing part of Phase 2.



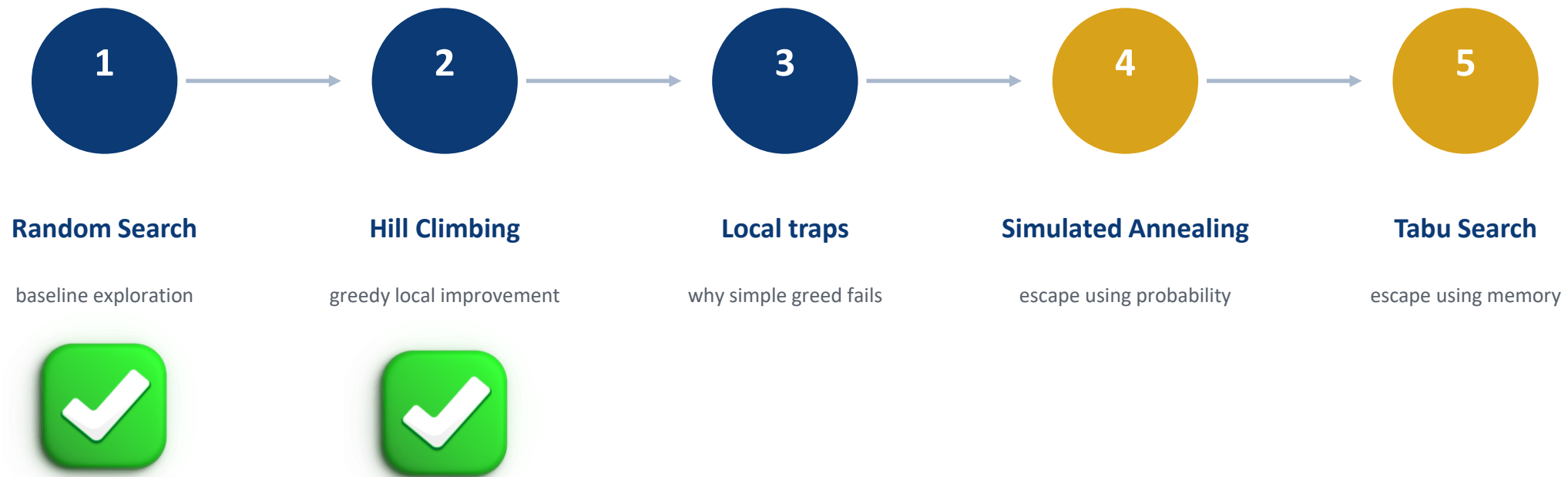


- We now finish the missing part of Phase 2.



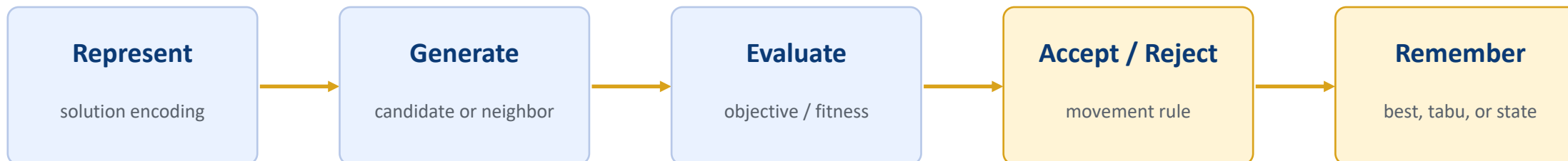


- We now finish the missing part of Phase 2.

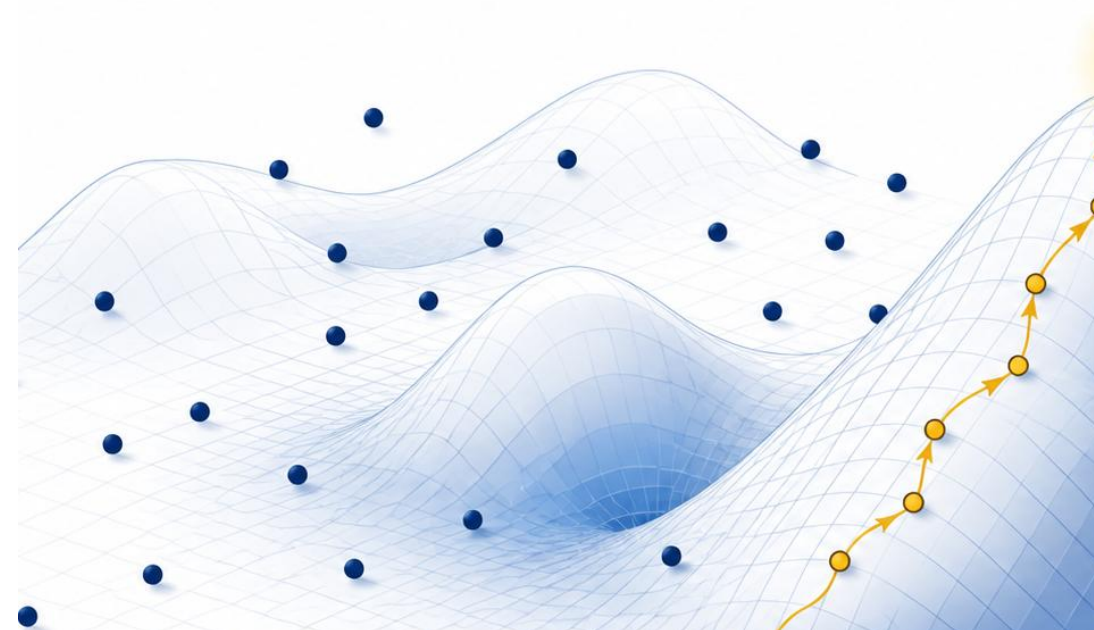




- Every algorithm in this section repeats the same loop:



- Random Search changes how we generate.
- Hill Climbing changes how we move.
- SA changes the acceptance rule.
- Tabu Search changes what we remember.

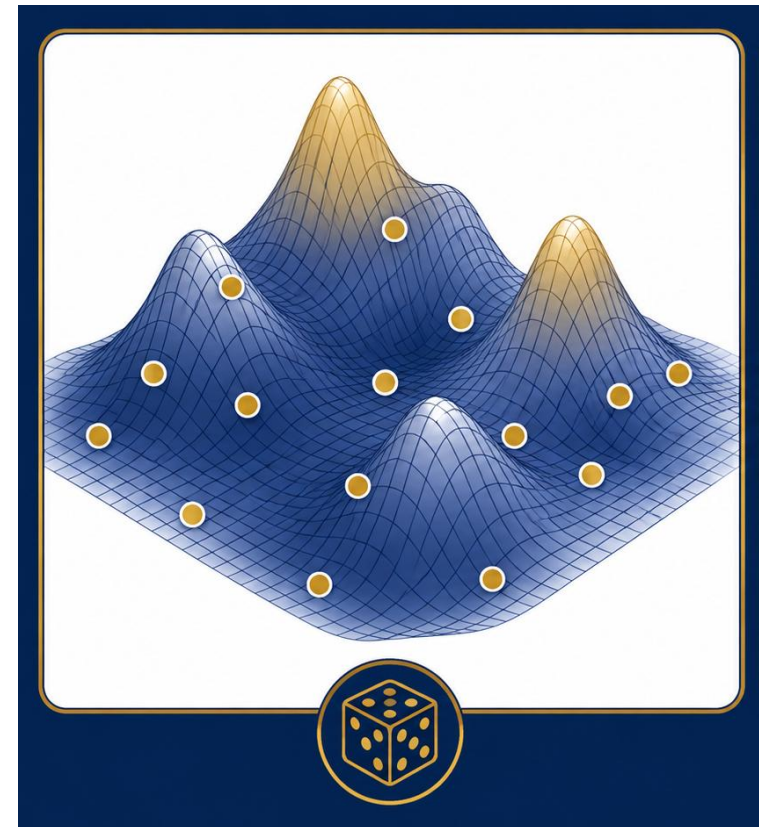




- Core idea
 - Generate many candidate solutions randomly.
 - Evaluate each candidate with $f(x)$.
 - Keep the best-so-far solution.
 - No direction, no neighborhood, no learning.

Generate → Evaluate → Keep best

Role in class: honest baseline





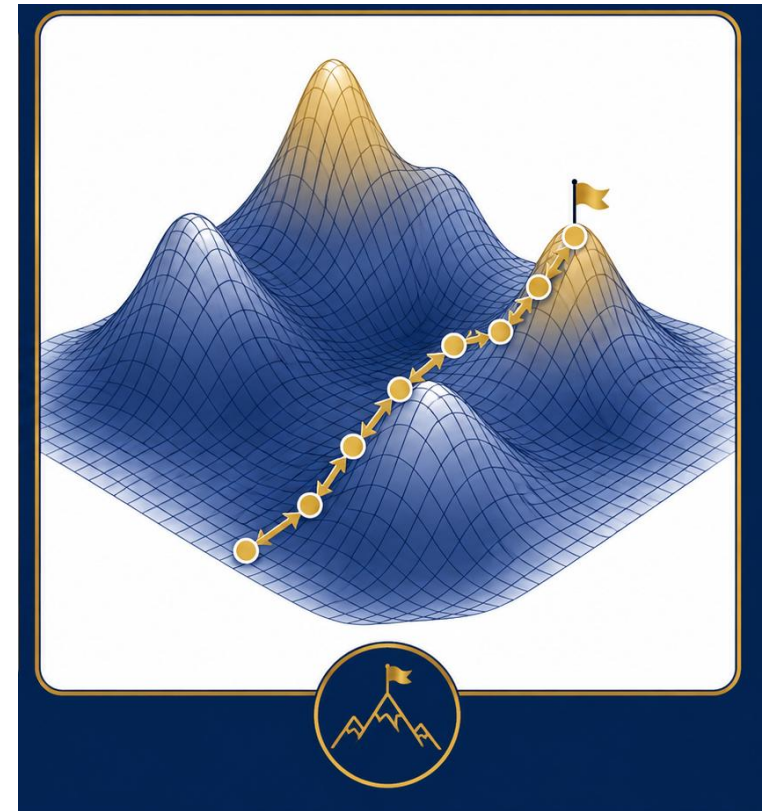
Hill Climbing: Greedy Local Improvement



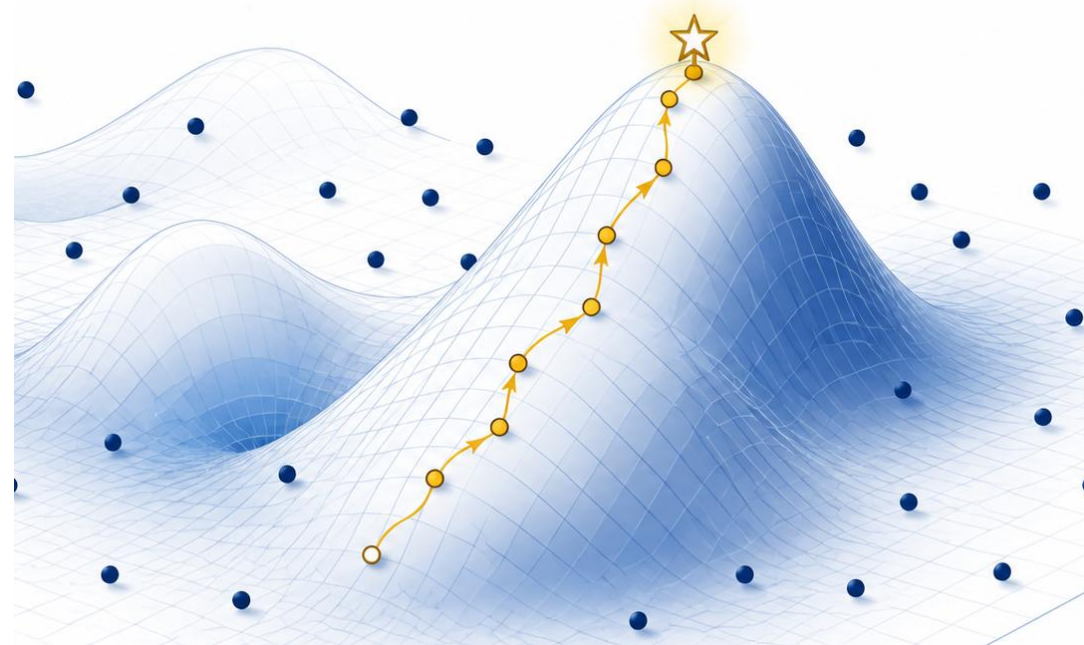
- Core idea
 - Start from one current solution x .
 - Generate neighbors using a small move.
 - Move only if the neighbor is better.
 - Stop when no improving neighbor exists.

Neighbor better? Move. Otherwise stop.

Fast, focused, and greedy



- **Hill Climbing** can stop even when better solutions exist elsewhere.
 - It only compares nearby solutions.
 - If all neighbors look worse, it stops.
 - Local information can be incomplete.
 - We need controlled escape mechanisms.





Probability and memory
are two classic solutions to local traps!

Simulated Annealing

Sometimes accept worse moves.

Tabu Search

Remember where not to go.



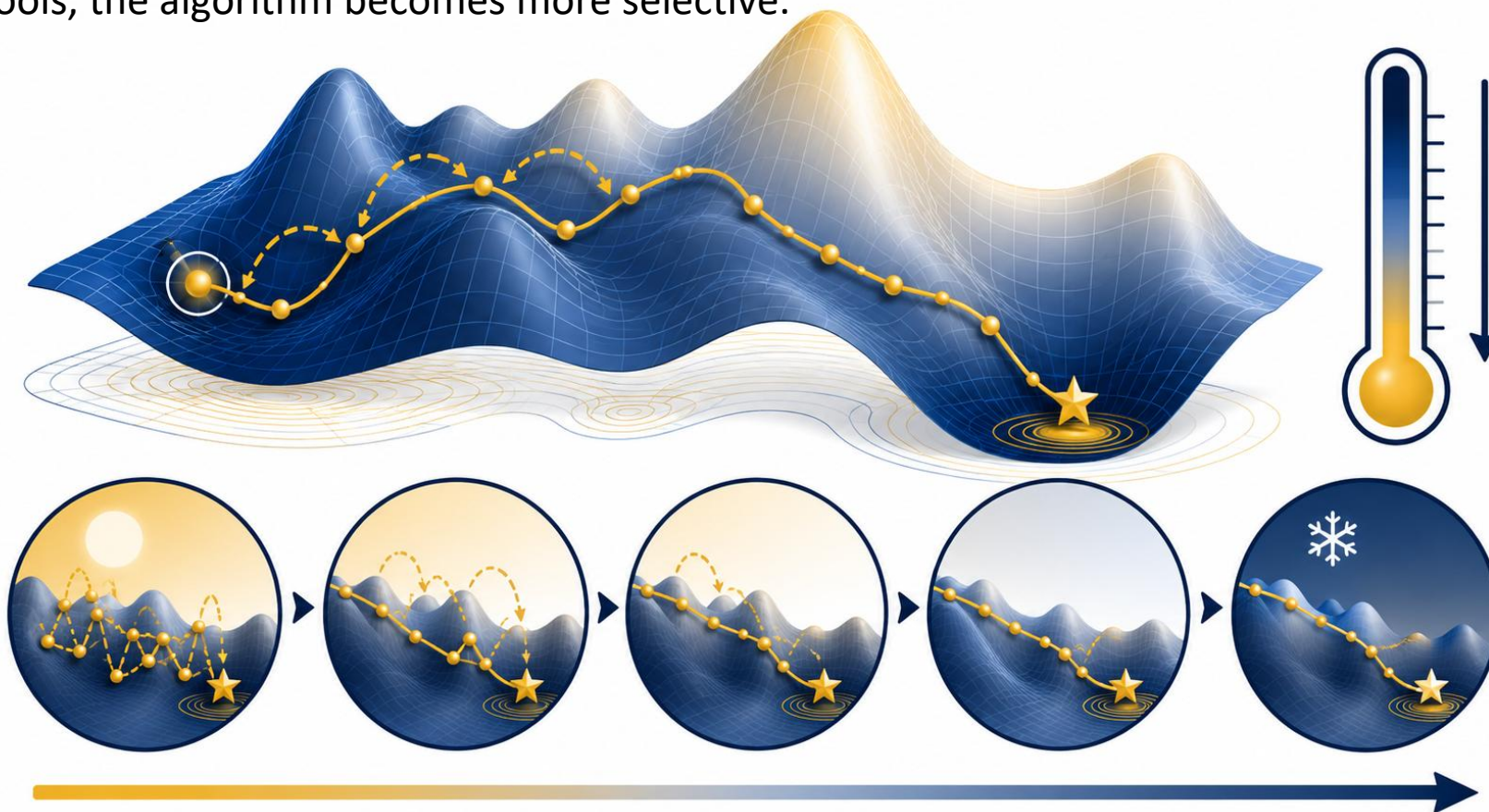
- Idea: start exploratory, then gradually become greedy.
 - Always accept better moves.
 - Sometimes accept worse moves.
 - The probability depends on temperature T
 - As T cools, the algorithm becomes more selective.



Simulated Annealing: Core Idea



- Idea: start exploratory, then gradually become greedy.
 - Always accept better moves.
 - Sometimes accept worse moves.
 - The probability depends on temperature T
 - As T cools, the algorithm becomes more selective.





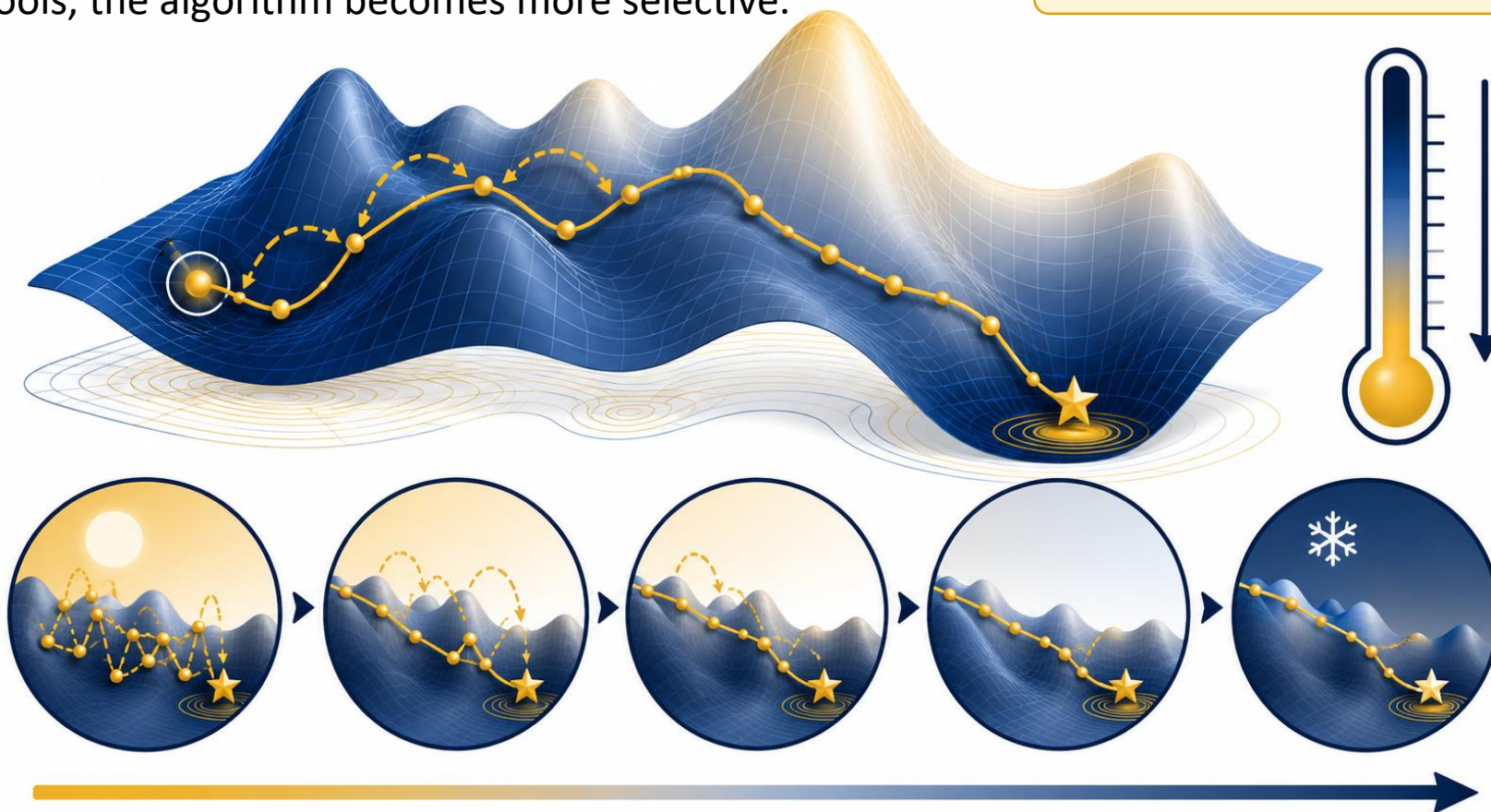
Simulated Annealing: Core Idea



- Idea: start exploratory, then gradually become greedy.

- Always accept better moves.
- Sometimes accept worse moves.
- The probability depends on temperature T
- As T cools, the algorithm becomes more selective.

High T : explore → Low T : refine





Generate an initial solution x
Set initial temperature T

Repeat until **stopping condition**:

Generate a neighbor x'

Compute $\Delta = f(x') - f(x)$

Accept if better, or **probabilistically** if worse

Reduce temperature T

Return best solution found

Cooling schedule controls the search behavior



Strengths

- Can escape local optima.
- Simple extension of local search.
- Works with many neighborhoods.
- Good for rugged search spaces.

Limitations

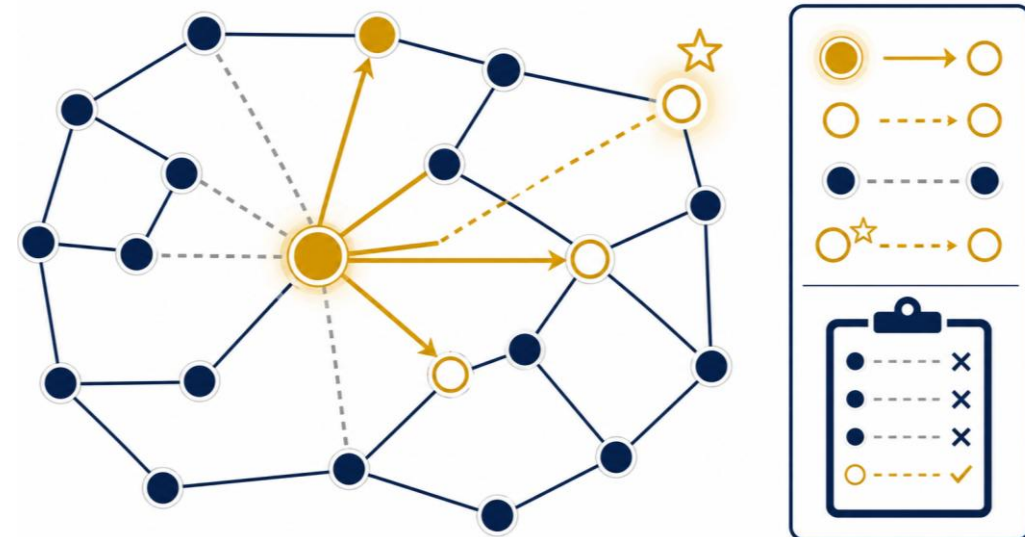
- Sensitive to initial temperature.
- Cooling schedule matters a lot.
- May be slow if cooling is conservative.
- No explicit memory of visited states.

SA is Hill Climbing with controlled willingness to make temporary mistakes



- Idea: improve locally, but use memory to avoid cycling.
 - Move to the best allowed neighbor.
 - Recent moves become tabu: temporarily forbidden.
 - Tabu list prevents immediate backtracking.
 - Aspiration can override tabu if the move is excellent.

Memory guides exploration





- Tabu **memory** usually stores moves, **not whole solutions!**

Current solution	The point where search is now
Neighborhood	Candidate moves around current solution
Tabu list	Recently used moves or attributes
Tabu tenure	How long a move stays forbidden
Aspiration	Override rule for exceptional moves

Example for TSP: after swapping cities B and D, immediately reversing the same swap can be tabu.

Generate an initial solution x
Initialize tabu list = empty

Repeat until **stopping condition**:

Generate neighbors of x

Remove tabu moves unless aspiration applies

Choose the best allowed neighbor x'

Move to x' and update best-so-far

Update tabu list and tabu tenure

Return best solution found

Best allowed move, not simply best move



Strengths

- Strong at avoiding short cycles.
- Uses problem-specific neighborhood design.
- Can move beyond local optima.
- Memory makes search more systematic.

Limitations

- Requires good tabu tenure choice.
- Memory design is problem-dependent.
- May still intensify too much without diversification.
- Can be more complex than SA.

Tabu Search is local search with a short-term memory

Algorithm	Main behavior	Escape mechanism	Best teaching role
Random Search	Broad sampling	None	Baseline exploration
Hill Climbing	Greedy improvement	None	Local exploitation
Simulated Annealing	Probabilistic local search	Worse moves at high T	Controlled randomness
Tabu Search	Memory-guided local search	Tabu list + aspiration	Avoid cycles

Message: each new method modifies one part of the same search loop.

Algorithm	Main behavior	Escape mechanism	Best teaching role
Random Search	Broad sampling	None	Baseline exploration
Hill Climbing	<u>Greedy</u> improvement	None	Local exploitation
Simulated Annealing	<u>Probabilistic</u> local search	Worse moves at high T	Controlled randomness
Tabu Search	<u>Memory-guided</u> local search	Tabu list + aspiration	Avoid cycles

Message: each new method modifies one part of the same search loop.



- 1 Run 5 iterations of Random Search
- 2 Run 5 moves of Hill Climbing
- 3 Allow one worse move as SA when T is high
- 4 Mark the reverse move as tabu for 2 turns
- 5 Compare best value and behavior

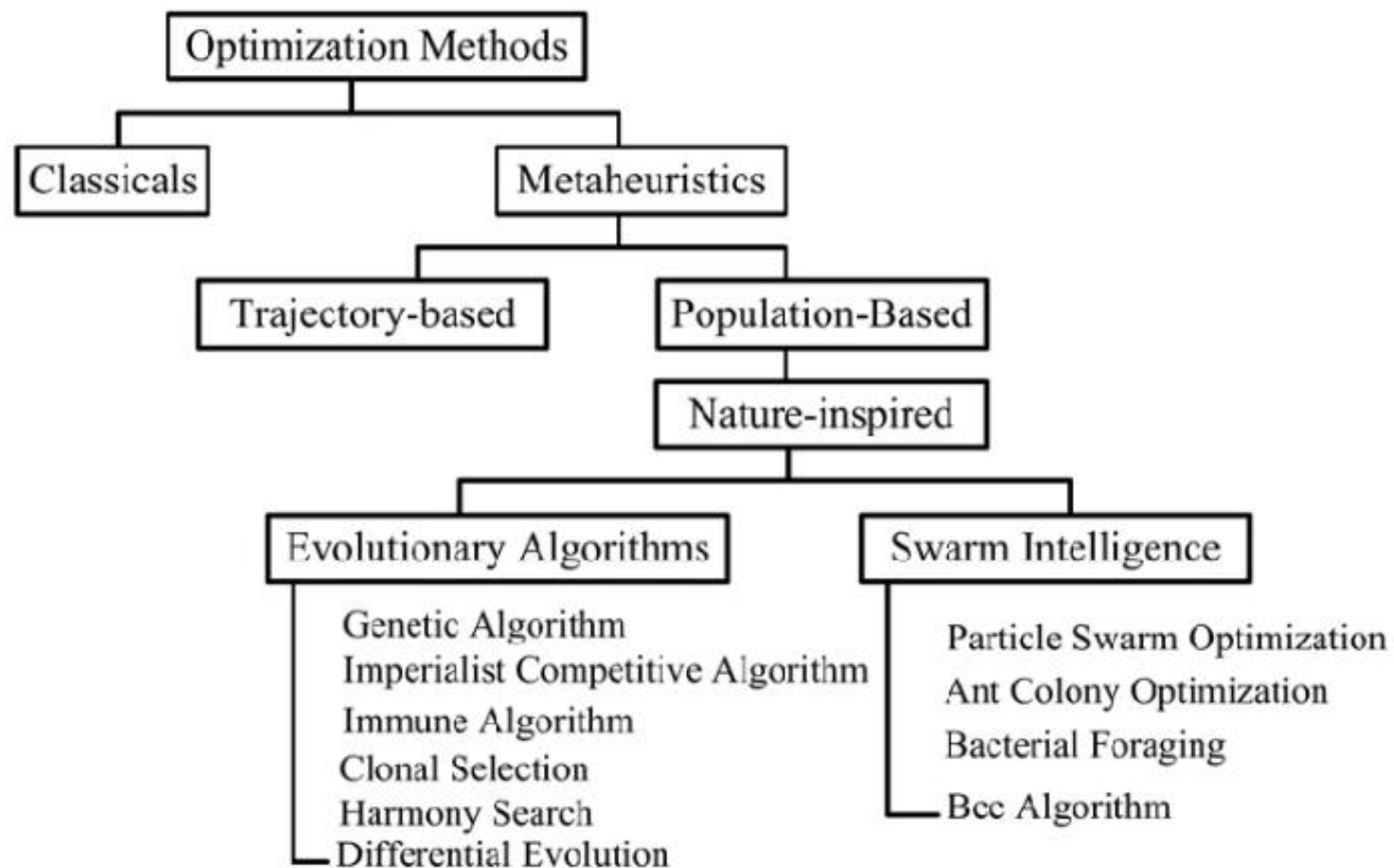
Discussion: Which method explored more? Which improved faster? Which avoided the local trap?

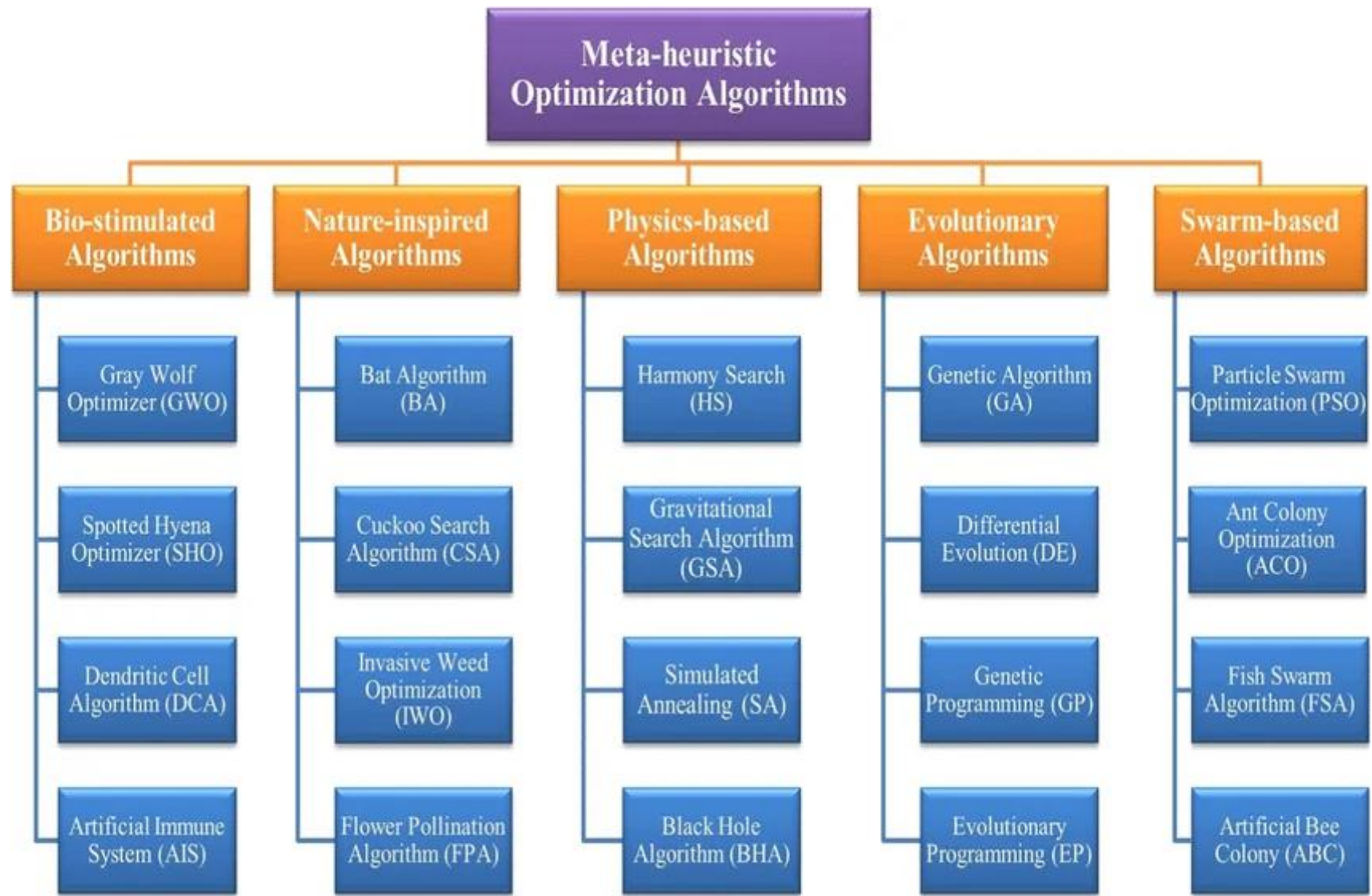


- Random Search gives a baseline for unguided exploration.
- Hill Climbing uses neighborhoods for greedy improvement.
- Local optima motivate escape mechanisms.
- Simulated Annealing escapes by probability and cooling.
- Tabu Search escapes by memory and forbidden moves.

Next phase: population-based algorithms

Now Phase 2 is complete.





https://www.researchgate.net/publication/333940328_A_comparative_review_of_meta-heuristic_approaches_to_optimize_the_SLA_violation_costs_for_dynamic_execution_of_cloud_services?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6Ii9kaXJlY3QiLCJwYVWdlIjoiX2RpcmVjdCJ9fQ

